



Research paper

Agentic AI for autonomous condition monitoring and predictive maintenance of marine vessels

Francesco Maione ^a,¹ Seher Suendam Arici ^{b,c}, Agus Hasan ^b,^{*} Modestas Sukarevicius ^b, Emre Akyuz ^{c,d}, Anniken Karlsen ^b

^a Politecnico di Bari, Via Edoardo Orabona 4, Bari, 70125, Italy

^b Norwegian University of Science and Technology, Larsgårdsvegen 2, Ålesund, 6025, Møre og Romsdal, Norway

^c Istanbul Technical University, Mah. Manastır Yolu Cad. No:1 -11-1, İstanbul, 34940, Türkiye

^d Azerbaijan State University of Economics, İstiqlaliyyət 22, Sabail, Baki, Baku, AZ1001, Azerbaijan

ARTICLE INFO

Dataset link: [Engine data \(Original data\)](#)

Keywords:

Agentic AI
Large language models
Marine vessels
Condition monitoring
Predictive maintenance
Transformer autoencoder

ABSTRACT

Modern marine vessels generate large volumes of high-frequency operational data, yet translating this data into reliable maintenance intelligence remains challenging. Existing solutions rely on threshold alarms, rule-based logic, or isolated machine learning models, which lack contextual reasoning, adaptability, and interpretability in safety-critical environments. This paper presents an Agentic AI architecture for condition monitoring and predictive maintenance of marine vessels. The proposed system integrates a Transformer-based Autoencoder for multivariate anomaly detection with a Large Language Model (LLM) that enables interactive, explainable analysis through structured tool-calling. The autoencoder learns normal operational patterns from high-dimensional time-series data and detects deviations associated with abnormal loading, inefficient generator utilisation, and potential equipment degradation. Rather than ingesting raw numerical streams, the LLM invokes domain-specific tools to retrieve structured measurements, compute derived metrics, and query anomaly outputs before generating responses. This mechanism mitigates hallucination risks and avoids the brittleness of fine-tuned models as system behaviour evolves. The approach is validated on 91 days of data from an offshore construction vessel with a hybrid diesel-electric propulsion system and six gensets under dynamic positioning constraints. Four fault scenarios (slow drift, load imbalance, temporary reduction, spikes) were all correctly detected, with prognostic lead times between 97 and 637 min.

1. Introduction

Marine vessels are undergoing a rapid digital transformation. Modern offshore vessels continuously generate high-frequency multivariate data through integrated automation systems, power management platforms, and propulsion control networks. These systems operate under strict safety, redundancy, and regulatory constraints, particularly during dynamic positioning (DP) operations where reliability is mission-critical. As propulsion and power architectures become increasingly electrified and software-defined, system behaviour emerges from complex interactions between generators, grids, control algorithms, and operational modes. Although large volumes of operational data are now

available, transforming this data into reliable and actionable maintenance intelligence remains a significant challenge. Existing monitoring solutions are typically based on threshold alarms, rule-based logic, or isolated machine learning models. Such approaches lack contextual reasoning, struggle to adapt to evolving system configurations, and provide limited interpretability in safety-critical environments. Furthermore, anomaly detection systems often operate independently from decision-support tools, leaving human operators to manually bridge the gap between detection and diagnosis. These limitations highlight the need for an autonomous, integrated framework that combines data-driven anomaly detection with contextual reasoning and explainable decision support.

^{*} Corresponding author.

E-mail address: agus.hasan@ntnu.no (A. Hasan).

URL: <https://www.ntnu.edu/employees/agus.hasan> (A. Hasan).

¹ The activities conducted by Francesco Maione were developed within the Fincantieri project “Wave 2 the Future” (W2F), funded by the European Union – NextGenerationEU. The W2F project is part of IPCEI Hy2Tech.

1.1. Motivation

Modern vessels operate as highly complex cyber–physical systems that integrate propulsion, power generation, distribution grids, automation platforms, and safety-critical control mechanisms (Chae, 2025). Offshore vessels, especially those operating under DP constraints, must continuously balance efficiency, redundancy, and operational safety. Diesel-electric and hybrid propulsion systems introduce additional layers of complexity, where multiple generator sets (gensets), split bus configurations, spinning reserve requirements, and rapidly varying load profiles interact in non-trivial ways (Di Bernardo et al., 2025). Despite the availability of high-frequency sensor data through Integrated Automation Systems (IAS), condition monitoring and maintenance planning remain largely reactive or rule-based. Traditional alarm systems detect threshold violations but fail to capture the subtle multivariate deviations that precede faults. Physics-based models, while accurate for specific subsystems, are difficult to scale across heterogeneous components and evolving operational modes. Data-driven anomaly detection methods have shown promise, but they often operate as isolated analytics modules without explainability, contextual reasoning, or interactive decision support.

At the same time, the emergence of Large Language Models (LLMs) has enabled new forms of human-machine interaction (Chen et al., 2025). However, directly applying LLMs to raw industrial time-series data introduces significant risks: hallucinated interpretations, outdated knowledge after system modifications, and a lack of grounding in real-time measurements. Fine-tuning models on historical datasets does not solve this problem, because vessel configurations, load patterns, and operational policies evolve over time. In safety-critical maritime environments, such brittleness is unacceptable. These limitations motivate a new paradigm: Agentic AI for autonomous condition monitoring and predictive maintenance. Rather than treating anomaly detection, data retrieval, and reasoning as separate components, an agentic architecture orchestrates them through structured tool invocation and real-time grounding. A Transformer-based autoencoder captures normal operational behaviour across high-dimensional signals, detecting deviations that may indicate inefficient loading, abnormal generator behaviour, or early-stage equipment degradation. An LLM then acts not as a predictor of raw sensor values, but as a reasoning layer that invokes computational tools to retrieve measurements, compute derived metrics (e.g., load factors, reserve margins), and interpret anomaly outputs before generating explanations or recommendations.

This integration addresses three core challenges in maritime maintenance:

- *Operational complexity*: marine power systems operate under varying weather, mission modes, and regulatory constraints, making static models insufficient.
- *Data richness without insight*: high-resolution sensor data is available, but actionable and contextual interpretation remains limited.
- *Trust and explainability*: safety-critical environments require traceable reasoning grounded in verifiable data sources.

By combining deep anomaly detection with tool-grounded language reasoning, the proposed Agentic AI framework aims to move from passive monitoring towards autonomous, interpretable, and adaptive predictive maintenance. Such a system not only detects deviations but also explains them in operational context, supports human operators in decision-making, and adapts to evolving system configurations without requiring continuous model retraining. Ultimately, this work is motivated by the need to bridge advanced machine learning with operational reliability in maritime systems, enabling intelligent vessels capable of safer, more efficient, and more sustainable operation.

1.2. Related works and research gap

The development of machine learning and deep learning algorithms has driven a paradigm shift in maintenance operations: scheduling is no longer time-based (preventive maintenance) (Susto et al., 2015) but chosen on the basis of the real state of health of the monitored system (condition-based and predictive maintenance) (Han et al., 2021a). Condition monitoring and predictive maintenance in marine vessels have traditionally relied on physics-based models, rule-based alarm systems, and expert-driven diagnostics (Cui et al., 2024). Critical subsystems such as diesel engines, generators, propulsion units, and auxiliary machinery are typically supervised using threshold-based alarms derived from manufacturer specifications and classification society regulations. While these approaches are dependable for detecting limit violations and ensuring regulatory compliance, they are fundamentally reactive. They often fail to capture subtle multivariate deviations across interacting subsystems, which may signal incipient faults or progressive degradation long before alarms are triggered.

To overcome these limitations, the maritime domain has increasingly adopted data-driven methodologies inspired by developments in industrial prognostics and health management. Statistical approaches such as Principal Component Analysis (PCA) and autoregressive modelling have been applied to fault detection in marine engines and power systems (Zhang et al., 2022). More recently, machine learning techniques, including Support Vector Machines (SVMs), Random Forests, and various neural network architectures, have been used for fault classification and remaining useful life (RUL) estimation (Alfarizi et al., 2022). Despite their promise, many of these supervised methods depend on labelled fault data, which is inherently scarce in maritime environments. The high reliability, built-in redundancy, and strict safety margins of marine systems mean that true failure events are infrequent, limiting the availability of representative fault datasets. Consequently, unsupervised and semi-supervised anomaly detection approaches have gained increasing attention. Autoencoder-based models, including Long Short-Term Memory (LSTM) networks and more recently Transformer-based architectures, have demonstrated a strong capability to learn normal operational behaviour from high-dimensional time-series data (Sun et al., 2024).

Within this family of methods, reconstruction algorithms are often successfully applied (Han et al., 2021b; Ellefsen et al., 2019). Reconstruction algorithms refer to the class of machine and deep learning models trained to reconstruct their inputs, that is, they produce outputs almost equal to the inputs. Differently from other approaches that rely on the comparison between healthy and faulty values (Maione et al., 2025b), the strength of reconstruction algorithms is that their training is not affected by the faults themselves. The traditional approach involves training a predictive algorithm, such as an LSTM or an SVM, to directly predict the RUL or the expected (healthy) sensor measurements, which are then compared with the acquired ones. The amount of drift between the measured variables and the predicted ones is correlated with specific faults. This approach works as long as the inputs of the predictive algorithm are not affected by faults; otherwise, the predictions are no longer reliable, and the drift and location of the faults become uncertain. In contrast, reconstruction algorithms turn this weakness into their strength. Condition-monitoring approaches based on reconstruction algorithms work as follows: the inputs, namely the sensor measurements, are processed and reconstructed through the hidden layers; since the outputs are expected to be equal to the inputs, the drift between them is used as an indicator of the state of health of the monitored system. This approach monitors the *global* health of the machines: since all sensor measurements are used as inputs, it is not necessary to identify robust variables that are unaffected by faults. The Variational Autoencoder and the LSTM-Autoencoder are the most widely applied algorithms for this kind of approach; however, the recent development of Transformers and their application to time-series prediction make them worth investigating for monitoring problems

(Bampoula et al., 2024). In particular, Transformers leverage the self-attention mechanism to capture long-range temporal dependencies and inter-sensor relationships within multivariate time series, thereby enabling predictions based on a broader contextual representation of the input data.

By modelling nominal system dynamics, these methods can identify deviations without requiring labelled fault examples, making them particularly suitable for complex marine applications. Nevertheless, existing work typically treats anomaly detection as a standalone analytical module: emphasis is placed on detection accuracy and reconstruction error metrics, while limited attention is given to operational integration, contextual interpretation, or decision-support mechanisms for crew and operators. The complexity increases further in diesel-electric and hybrid propulsion systems. Such vessels operate with multiple generator sets, split bus configurations, redundancy requirements, spinning reserve constraints, and DP regulations that significantly influence load distribution and system behaviour (Kolodziejski and Michalska-Pozoga, 2023). Prior research in this area has largely focused on generator scheduling optimisation, load-sharing strategies, spinning reserve management, and fuel efficiency improvements (Park and Kim, 2024). Approaches based on rule-based power management, optimisation algorithms, and model predictive control have been proposed to enhance generator utilisation and reduce fuel consumption. While these strategies improve operational efficiency, they typically assume clearly defined system states and do not explicitly address autonomous condition monitoring or early degradation detection. Moreover, optimisation frameworks generally operate independently of anomaly detection systems and lack mechanisms for natural language explanation or interactive reasoning. As marine systems become increasingly data-rich and partially autonomous, the absence of integrated monitoring and reasoning capabilities becomes a critical limitation.

In parallel, the rapid advancement of LLMs has introduced new opportunities for interactive analytics, knowledge retrieval, and decision support in engineering environments (Alkayyal et al., 2025). LLMs have been explored for maintenance report generation, technical documentation assistance, and conversational interfaces for industrial databases. However, directly applying LLMs to raw sensor streams presents substantial challenges. LLMs are not inherently designed for numerical time-series analysis and may produce hallucinated or ungrounded interpretations when prompted with raw measurements (Heo et al., 2025). Although fine-tuning on historical operational data can improve domain adaptation, such models remain brittle: they can quickly become outdated when vessel configurations, operational policies, or environmental conditions evolve. In safety-critical maritime environments, ungrounded reasoning and a lack of traceability are unacceptable. To address these limitations, recent developments in tool-augmented LLMs and agentic architectures have demonstrated the value of structured tool-calling (Wang et al., 2025). Rather than reasoning directly over raw numerical inputs, the language model invokes external computational tools, databases, or analytical modules, and subsequently reasons over verified outputs. This paradigm enhances grounding, traceability, and adaptability. While tool-augmented architectures have shown promise in general AI systems and software engineering applications, their systematic application to maritime condition monitoring and predictive maintenance remains largely unexplored. Bridging deep anomaly detection, marine power system monitoring, and tool-grounded language reasoning therefore represents a significant and timely research opportunity.

Despite substantial progress in data-driven anomaly detection, marine power optimisation, and language-model-based interfaces, important gaps remain. Current anomaly detection models are typically deployed as standalone analytical components, disconnected from contextual reasoning and decision-support mechanisms. As a result, operators must manually interpret anomaly scores and relate them to operational constraints, limiting automation and scalability. Although

deep learning models, including Transformer architectures, offer powerful multivariate representation learning, they often lack mechanisms for interactive explanation within operational contexts. At the same time, the application of LLMs in industrial environments introduces new challenges. Directly exposing LLMs to raw sensor streams risks hallucinated or ungrounded interpretations, while fine-tuning alone does not ensure robustness as system configurations and operational behaviours evolve. Moreover, existing maritime monitoring architectures rarely integrate anomaly detection, metric computation, contextual reasoning, and interactive explanation within a unified autonomous framework. Finally, there is limited validation of intelligent monitoring systems on complex hybrid diesel-electric vessels operating under DP constraints, where redundancy requirements and grid splitting significantly influence system behaviour. Together, these limitations highlight the need for an integrated, grounded, and operationally validated approach to autonomous maritime condition monitoring.

1.3. Contributions of this paper

The aim of this work is to demonstrate the feasibility of deploying an LLM-based Agentic AI framework for condition monitoring in the maritime domain, which is one of the sectors most reluctant to adopt new digital technologies, rather than to propose a new anomaly-detection algorithm to be ranked against existing ones. The contribution lies in the integration of unsupervised detection, prognostic reasoning, and tool-grounded language interaction within a single traceable architecture suited to safety-critical constraints. In response to the gaps identified above, this paper makes the following contributions:

- *Agentic AI architecture for maritime predictive maintenance.* A unified agentic framework is proposed that integrates anomaly detection, computational tools, and language-based reasoning for autonomous condition monitoring of marine vessels. Unlike prior work, detection and decision support are not treated as separate modules but are orchestrated within a single autonomous pipeline.
- *Transformer-based unsupervised anomaly detection for marine power systems.* A multivariate Transformer Autoencoder is developed to learn normal operational behaviour from high-frequency vessel time-series data without requiring labelled fault datasets. This addresses the scarcity of failure data in maritime environments and moves beyond threshold-based and supervised approaches.
- *Tool-grounded LLM reasoning for safety-critical systems.* Structured tool-calling is introduced to ground LLM reasoning in verified, real-time computational outputs. Unlike naive LLM applications that ingest raw sensor data or rely solely on fine-tuning, our approach mitigates hallucination risks and ensures traceable, data-backed responses.
- *Validation on a real hybrid diesel-electric offshore vessel.* The framework is validated using 91 days of operational data from an offshore construction vessel with six gensets and a split 690 V main switchboard, covering four representative fault scenarios including sustained (slow drift, load imbalance) and transient (temporary reduction, spikes) anomalies.

The remainder of this paper is organised as follows. Section 2 presents the conceptual foundations of the proposed Agentic AI framework. Section 3 details the methodology, including the Transformer Autoencoder, the LLM tool-calling layer, and the case-study dataset. Section 4 describes the use cases and simulated fault scenarios. Section 5 reports and discusses the results, and Section 6 concludes the paper and outlines future research directions.

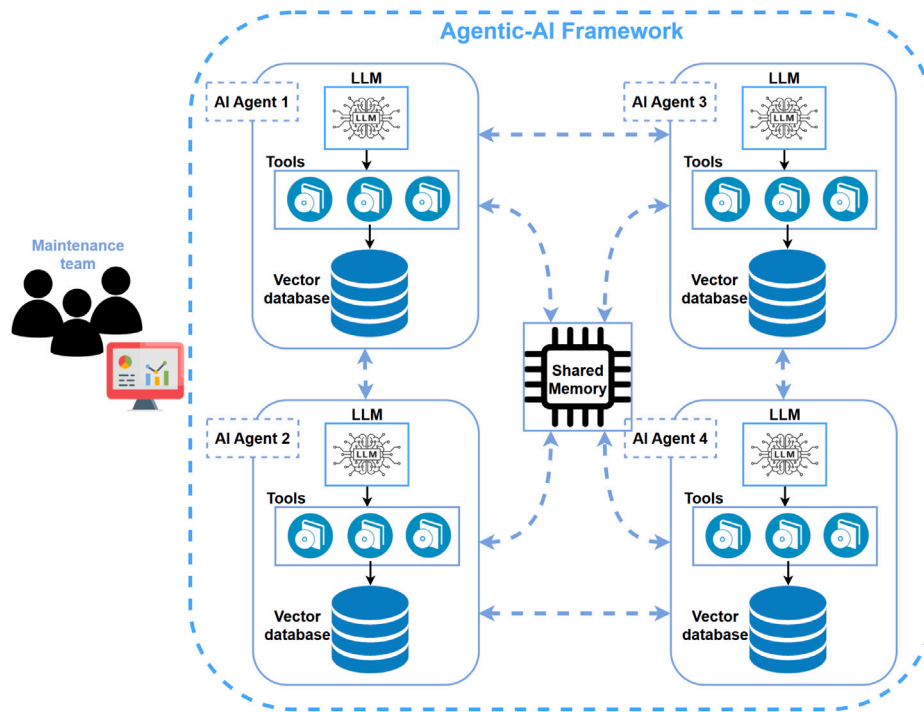


Fig. 1. Conceptual architecture of the proposed Agentic AI framework. The framework is organised as a multi-agent system in which each AI agent integrates an LLM, domain-specific tools, and a vector database, coordinated through a shared memory module to enable grounded reasoning, collaborative intelligence, and explainable decision support for maritime condition monitoring and predictive maintenance.

2. Conceptual foundations of Agentic AI

The conceptual foundation of the proposed Agentic AI framework is the transition from isolated analytical models to coordinated, goal-directed intelligent agents capable of perception, reasoning, action, and collaboration. Rather than treating anomaly detection, data retrieval, and explanation as independent modules, the framework organises intelligence as an ecosystem of interacting agents operating within a unified architectural boundary. As illustrated in Fig. 1, the Agentic-AI Framework encloses multiple AI agents, each equipped with an LLM, domain-specific tools, and access to both local and shared memory resources. Together, these components form a distributed yet coherent cognitive system designed for autonomous condition monitoring and predictive maintenance.

At the core of each AI agent lies an LLM, which serves as the cognitive controller rather than a standalone predictor. The LLM does not directly process raw time-series sensor data; instead, it orchestrates reasoning by invoking structured tools, retrieving contextual information from memory, interpreting analytical outputs, and generating explainable responses. This distinction is fundamental. In safety-critical maritime systems, ungrounded reasoning over raw numerical streams is unacceptable. By positioning the LLM as a supervisory reasoning layer, the framework ensures that all interpretations are based on verified computational results rather than probabilistic language inference alone. Beneath the LLM within each agent resides a layer of domain-specific tools. These tools represent deterministic or statistical modules, such as anomaly detection models, performance metric calculators, trend analysers, and system-state interpreters. Conceptually, this layer anchors intelligence in computation. When an agent is tasked with evaluating vessel condition or responding to a maintenance query, it decomposes the task into structured sub-operations and invokes the appropriate tools. The outputs, such as anomaly scores, load factors, reserve margins, or historical comparisons, are then returned to the LLM for contextual interpretation. This tool-augmented reasoning paradigm ensures traceability, robustness, and operational reliability, particularly in environments governed by strict safety and regulatory requirements.

Each agent is also connected to a vector database that functions as long-term semantic memory. This memory layer stores embeddings of historical operational states, maintenance records, anomaly descriptions, and domain knowledge. Through similarity-based retrieval, the agent can compare current system behaviour to past patterns, identify recurring inefficiencies, and distinguish novel deviations from known operational modes. Memory transforms the agent from a stateless processor into a learning entity capable of contextual adaptation. Rather than retraining models for every change in system configuration, the framework accumulates knowledge over time and uses retrieval to enhance reasoning continuity.

Beyond individual agents, the architecture introduces a shared memory module at the centre of the framework. This shared memory acts as a global knowledge substrate accessible by all agents, enabling synchronisation of system understanding, task coordination, and collective situational awareness. When one agent detects an anomaly or derives a performance insight, that information can be stored centrally and accessed by others. This design promotes distributed specialisation without sacrificing coherence. For example, a power-system monitoring agent may detect abnormal generator loading, and a maintenance-planning agent can use that information to assess risk implications or scheduling adjustments. The shared memory thus facilitates collective intelligence, where insights are not siloed but integrated across functional domains. The multi-agent structure shown in Fig. 1 reflects deliberate modular specialisation. Each AI agent can focus on a distinct subsystem or analytical objective, such as propulsion diagnostics, power management efficiency, operational compliance, or maintenance optimisation. Instead of relying on a monolithic intelligence layer, the framework distributes expertise across agents while maintaining communication pathways through shared memory. This modularity enhances scalability and resilience: new agents can be introduced as vessel complexity grows, and individual agents can be updated independently without disrupting the overall architecture.

Interaction with human operators is also embedded conceptually within the framework. The maintenance team interfaces with the system through LLM-mediated communication, receiving interpretable explanations, summaries, and recommendations derived from structured

tool outputs. In this sense, the LLM acts as both a reasoning engine and an interface translator, converting complex multivariate analytical results into actionable insights. Human oversight remains central, but the cognitive burden of interpreting raw anomaly scores and cross-referencing operational constraints is significantly reduced. All agents, tools, and memory modules operate within a defined architectural perimeter, ensuring security, governance, and traceability. Autonomy in this context does not imply unrestricted behaviour; rather, it denotes structured, rule-aligned intelligence operating under explicit constraints. This is particularly important in maritime systems where compliance with classification society regulations and safety protocols is mandatory. Collectively, the conceptual principles embodied in the framework include grounded reasoning through tool invocation, separation of computation and language interpretation, persistent memory for contextual continuity, distributed specialisation across agents, and explainable outputs for human collaboration. The architecture reflects a shift from reactive alarm-based supervision towards proactive, coordinated, and interpretable intelligence.

In complex hybrid diesel-electric vessels operating under DP constraints, system behaviour emerges from intricate interactions between generators, grids, propulsion units, and regulatory rules. Addressing such complexity requires more than isolated anomaly detection models: it requires agents capable of observing system states, invoking computational analyses, contextualising outputs against historical memory, collaborating with other agents, and communicating findings to human operators. The conceptual foundation of Agentic AI, as represented in Fig. 1, operationalises this vision by integrating perception, reasoning, memory, and collaboration within a unified multi-agent ecosystem. In the present work, this general conceptual framework is instantiated through two specialised agents: a *monitoring agent*, responsible for real-time anomaly detection based on Transformer reconstruction error, and a *predictive agent*, responsible for prognostic reasoning based on linear extrapolation of the error trend. Both agents share access to the same tool suite and interact with the operator through a unified LLM interface, as detailed in Section 3.

For the purposes of this work, an *agent* is defined as a computational entity that (i) *perceives* the state of its environment through structured observations, (ii) *reasons* about that state in relation to a goal, (iii) *autonomously decides which actions to take*, which computational tools to invoke and in what order, rather than executing a fixed predetermined pipeline, and (iv) *acts* by producing grounded outputs that inform subsequent decisions. Under this definition, the monitoring and predictive functions qualify as agents because the LLM is not constrained to a fixed sequence of operations: given an operator query and the current context, it selects and chains tool invocations dynamically, interprets their structured results, and determines whether further actions are required before responding. The two agents are distinguished by their goals and characteristic action repertoires, which are the real-time state assessment via reconstruction-error tools for the monitoring agent, and prognostic trend extrapolation for the predictive agent, while sharing a common tool suite and memory. This framing motivates the term *Agentic AI* in preference to narrower descriptions. It differs from *LLM-assisted monitoring*, in which the language model plays a passive, explanatory role over outputs generated by a fixed analytics pipeline, because here the model actively decides the analytical actions to perform. It differs from a *Retrieval-Augmented (RAG) system*, which retrieves unstructured text passages by semantic similarity and conditions generation on them, whereas our agents invoke typed functions with defined schemas and reason over structured, computed results rather than retrieved documents. The closest description is a *tool-augmented LLM*, and indeed tool use is a core mechanism of the framework; the agentic character arises from the additional properties of goal-directed autonomy in action selection, a perceive–reason–act loop with iterative tool chaining, and persistent memory enabling contextual continuity across interactions. We note, however, that the present implementation is a deliberately minimal, two-agent instantiation of the more general multi-agent architecture described above (Fig. 1); richer forms of inter-agent coordination and autonomy are a direction for future work rather than a claim of the current system.

3. Methodology

3.1. Reconstruction error for condition monitoring

The proposed methodology addresses the main challenges of intelligent condition monitoring of a ship propulsion plant and uses this information to forecast the expected time to threshold crossing. Plant monitoring is performed by applying a Transformer algorithm trained to reconstruct the input variables; the reconstruction error is then evaluated as the metric that indicates the state of health of the ship's propulsion plant. If the error exceeds a fixed threshold, an anomaly is detected. The next step is to investigate how to use this error to move from a pure monitoring approach to a predictive one. Traditional reconstruction algorithms are excellent for *current* anomaly detection, that is, for providing information about how and how well the machine is operating; however, they provide no information on the future state of health. To overcome this limitation, the reconstruction errors evaluated step by step are stored and interpolated through linear regression. In this way, the predicted time of intersection between the linear regression extrapolation and the fault threshold is used as an early-warning estimate of the time remaining before the anomaly threshold is crossed. We emphasise that this constitutes early anomaly forecasting rather than a validated estimate of Remaining Useful Life (RUL): it extrapolates the observed error trend to a predefined threshold and does not model the full physical degradation trajectory, which is generally non-linear. The last open point is the evaluation of a suitable threshold for anomaly detection. The threshold is defined by computing the reconstruction error on the healthy dataset (it is worth noting that, due to noise and intrinsic training errors, the outputs will never be exactly equal to the inputs, even when the inputs are healthy); the maximum value is then taken and increased by a safety margin (here, 25%) to avoid false alarms.

The proposed framework for applying the introduced monitoring techniques comprises the following steps:

1. healthy data from the monitored machines are acquired;
2. the Transformer is trained to reconstruct the healthy sensor measurements acquired in the previous step;
3. the reconstruction error is evaluated on the healthy data and the anomaly detection threshold is computed;
4. the Transformer is deployed for the online monitoring of the machines of interest;
5. the reconstruction errors are stored, and a linear regression is fitted to them;
6. if the computed current error or the predicted future error exceeds the fixed threshold, an alarm is flagged.

Algorithm 1 and Fig. 2 further clarify the development of the proposed framework.

Once the main framework has been described, the remainder of this subsection addresses in more detail the Transformer architecture employed in the current work.

The Transformer architecture, originally proposed by Vaswani et al. (2017) for sequence-to-sequence modelling in natural language processing, has more recently been adopted for time-series analysis due to its ability to capture long-range temporal dependencies through self-attention mechanisms. In this work, a Transformer Autoencoder is employed as the reconstruction algorithm within the monitoring framework described above. The architecture follows an encoder–decoder structure that maps multivariate time-series windows to a compressed latent representation and reconstructs them back to the original input space. Given an input window $\mathbf{x} \in \mathbb{R}^{T \times F}$ with $T = 120$ timesteps and $F = 16$ sensor variables, the processing pipeline is:

$$\mathbf{x} \xrightarrow{\text{Linear}} \mathbf{h} \xrightarrow{\text{PE}} \mathbf{h}' \xrightarrow{\text{Encoder}} \mathbf{z} \xrightarrow{\text{Decoder}} \mathbf{h}'' \xrightarrow{\text{Linear}} \hat{\mathbf{x}}, \quad (1)$$

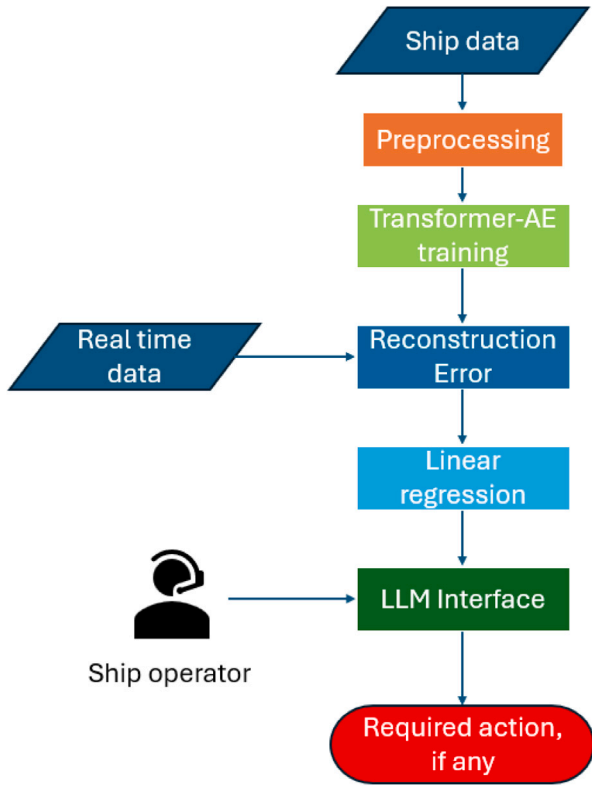


Fig. 2. Block diagram of the proposed framework, from raw ship data acquisition and pre-processing to the LLM-based interface that provides the ship operator with actionable recommendations.

where PE denotes sinusoidal positional encoding, $\mathbf{z} \in \mathbb{R}^{T \times d}$ is the latent representation with $d = 64$, and $\hat{\mathbf{x}} \in \mathbb{R}^{T \times F}$ is the reconstruction. The window size of 120 samples corresponds to 10 min of operational data at the 5-second sampling interval of the vessel's IAS. This duration was chosen to capture sufficient temporal context for the model to learn the dynamic patterns that characterise normal operations, including propulsion mode transitions, manoeuvring sequences, and load redistribution events.

The first linear layer projects each 16-dimensional feature vector into a 64-dimensional embedding space ($d_{\text{model}} = 64$). Since the self-attention mechanism is inherently permutation-invariant and does not encode sequential ordering, sinusoidal positional encoding (Vaswani et al., 2017) is added to the embedded representation to inject temporal position information:

$$\text{PE}_{(p,2i)} = \sin\left(\frac{p}{10000^{2i/d}}\right), \quad \text{PE}_{(p,2i+1)} = \cos\left(\frac{p}{10000^{2i/d}}\right), \quad (2)$$

where p is the position index within the window and i is the dimension index. This deterministic encoding allows the model to distinguish between identical sensor readings occurring at different points within the observation window, which is essential for learning temporal patterns such as ramp-up sequences and transient load events.

The encoder consists of two stacked Transformer encoder layers. Each layer comprises a multi-head self-attention sublayer followed by a position-wise feed-forward network, with residual connections and layer normalisation applied around each sublayer. The multi-head attention mechanism allows the model to attend to different temporal positions simultaneously through multiple attention heads:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad (3)$$

Algorithm 1: Training, deployment, and prognostic reasoning of the Transformer-based condition monitoring scheme.

Result: Training and application of the Transformer for monitoring problems.

Initialisation:

Acquired data matrix X : healthy sensor measurements from the monitored machines;

Training:

X is split into training, validation, and test sets:

$X_{tr}, X_{va}, X_{te} = X.\text{divide}(0.70, 0.15, 0.15)$;

Transformer.Training(input= X_{tr} , output= X_{tr});

Performance.Evaluation(output= $\hat{X}_{tr}$, real.output= X_{tr} , metrics= MSE);

Validation:

Transformer.Prediction(input= X_{va});

Performance.Evaluation(output= $\hat{X}_{va}$, real.output= X_{va} , metrics= MSE);

Test:

Transformer.Prediction(input= X_{te});

Performance.Evaluation(output= $\hat{X}_{te}$, real.output= X_{te} , metrics= MSE);

Threshold Definition:

Transformer.Prediction(input= $X_{healthy}$);

$Error_{healthy}(t)$: reconstruction error computation;

$e_{max} = Error_{healthy}.\text{max}()$: maximum reconstruction error;

$e_{threshold} = e_{max} + 0.25 e_{max}$;

Application to the monitored machine:

for each discrete time $t = k \Delta T$ do

$X(t)$: the acquired sensor measurements;

Transformer.Prediction(input= $X(t)$);

$e(t)$: the reconstruction error;

if $e(t) > e_{threshold}$ then

 a fault is detected;

end

else

$e(t)$ is stored in the vector $\eta(t)$;

 after Δt the acquired error vector $\eta(t)$ is used to fit a

 linear regression: $e_{predicted}(t) = m \eta(t) + c$;

 if $e_{predicted}(t) > e_{threshold}$ then

 a fault is predicted;

 end

end

end

where each of the $h = 4$ heads computes scaled dot-product attention independently:

$$\text{head}_j = \text{softmax}\left(\frac{QW_j^Q(KW_j^K)^T}{\sqrt{d_k}}\right) V W_j^V, \quad (4)$$

with key dimension $d_k = d_{\text{model}}/h = 16$. This multi-head formulation enables the model to simultaneously capture different types of temporal relationships within the sensor data, for example, short-range correlations between consecutive thruster activations and longer-range dependencies between generator loading and propulsion demand. The feed-forward sublayer within each encoder layer applies two linear transformations with a ReLU activation, $\text{FFN}(\mathbf{z}) = \max(0, \mathbf{z}W_1 + b_1)W_2 + b_2$, with a hidden dimension of 256.

The decoder mirrors the encoder architecture with two Transformer decoder layers. Each decoder layer contains a self-attention sublayer, a cross-attention sublayer that attends to the encoder output $\mathbf{z}$, and a feed-forward sublayer. During reconstruction, the decoder receives

Table 1
Transformer autoencoder hyperparameters.

Parameter	Value	Rationale
Input features F	16	All monitored IAS variables
Window size T	120	10 min at 5 s sampling
Training stride	12	1 min between windows
d_{model}	64	Embedding dimension
Attention heads h	4	Multi-head self-attention
Encoder/decoder layers	2	Transformer stack depth
Feed-forward dimension	256	FFN hidden size
Dropout	0.1	Regularisation
Training epochs	50	With best-checkpoint selection
Learning rate η	10^{-4}	AdamW optimiser
Weight decay λ	10^{-5}	L_2 regularisation
Batch size	512	Training mini-batch

the encoder's latent representation as both its memory input and its target sequence. This design choice means that the reconstruction is conditioned entirely on the compressed representation, forcing the encoder to retain sufficient information about normal operational patterns. Dropout with probability $p = 0.1$ is applied throughout the model for regularisation, and all weight matrices are initialised using Xavier uniform initialisation (Glorot and Bengio, 2010) to ensure stable gradient flow at the start of training.

The model is trained in a fully unsupervised manner using only data from normal vessel operations. Consistently with the framework introduced in Algorithm 1, the training data is assumed to consist of healthy measurements, and no labelled fault examples are required. Sliding windows of $T = 120$ timesteps are extracted from the normalised training data with a stride of 12 samples, corresponding to 1-minute intervals between consecutive windows. This overlapping windowing strategy yields a large number of training sequences that capture diverse operational states while maintaining computational efficiency. The reconstruction loss is the Mean Squared Error (MSE) between the input and its reconstruction:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2. \quad (5)$$

Optimisation is performed using the AdamW algorithm (Loshchilov and Hutter, 2019) with an initial learning rate of $\eta = 10^{-4}$ and weight decay $\lambda = 10^{-5}$ for L_2 regularisation. Gradient norms are clipped to a maximum of 1.0 to prevent exploding gradients, which can occur during early training when the model has not yet learned stable reconstruction patterns. A ReduceLROnPlateau scheduler monitors the validation loss and halves the learning rate after 5 consecutive epochs without improvement, allowing the optimiser to escape shallow local minima during later stages of training. The model is trained for a maximum of 50 epochs with a batch size of 512, and the checkpoint corresponding to the lowest validation loss is retained for deployment.

At inference time, the reconstruction error serves as the anomaly indicator, consistently with the monitoring framework introduced above. The per-window anomaly score is computed as the mean squared error averaged across all features and timesteps:

$$s(\mathbf{x}) = \frac{1}{T \cdot F} \sum_{t=1}^T \sum_{f=1}^F (x_{t,f} - \hat{x}_{t,f})^2. \quad (6)$$

In addition to the aggregate score, per-feature reconstruction errors are retained. This decomposition is particularly valuable in the maritime context, as it enables operators and the downstream LLM reasoning layer to identify which specific variables contribute most to an elevated anomaly score, rather than receiving a single opaque alarm. For instance, if the reconstruction error is dominated by contributions from Bus1 load variables, the system can direct attention to the corresponding generator and switchboard subsystems. The key hyperparameters of the Transformer Autoencoder are summarised in Table 1.

The choice of a Transformer Autoencoder over the more common LSTM Autoencoder and classical PCA reconstruction baselines is motivated by the characteristics of the data rather than by a claim of universal superiority. PCA-based reconstruction assumes an approximately linear subspace and captures only instantaneous cross-sensor correlations, so it is poorly suited to the non-linear, mode-dependent behaviour of a hybrid diesel-electric plant. LSTM Autoencoders model temporal dependencies recurrently but propagate information sequentially, which can attenuate long-range interactions between, for example, generator loading and delayed propulsion demand across a 120-sample window. The self-attention mechanism of the Transformer instead relates all timesteps directly, capturing both short-range correlations (e.g., consecutive thruster activations) and long-range multivariate dependencies within a single attention operation, which is well matched to the multi-mode operational profile described in the dataset. A controlled quantitative comparison against these baselines under identical conditions is left to future work, as discussed in Section 6.

3.2. LLM for interactive monitoring operations

While the reconstruction-based monitoring framework described in the previous subsection provides quantitative anomaly detection and prognostic capabilities, the interpretation of these outputs in operational context remains a significant challenge. An elevated reconstruction error affecting bus load variables, for instance, may have different implications depending on whether the vessel is operating in DP mode, transiting between locations, or performing crane operations. Similarly, a gradual upward trend in the anomaly score may indicate sensor drift, progressive equipment degradation, or a legitimate change in the operational profile. In current practice, bridging the gap between numerical anomaly outputs and actionable maintenance decisions falls entirely on human operators, who must mentally cross-reference detection results with system knowledge, operational constraints, and regulatory requirements. To address this limitation, the proposed framework integrates an LLM that provides interactive, natural language access to all monitoring outputs, enabling operators to query the system conversationally and to receive contextualised explanations grounded in verified data.

A fundamental design decision underlying the LLM integration is that the language model does not process raw sensor data directly. As discussed in the motivation and related work sections of this paper, feeding multivariate time-series arrays into a language model introduces substantial risks of hallucinated numerical interpretations, because LLMs are not inherently designed for precise arithmetic over high-dimensional numerical inputs (Huang et al., 2025). An alternative approach would be to fine-tune the language model on historical vessel operational data; however, this introduces a different form of brittleness. When vessel configurations change, for example when a generator is taken offline for maintenance or a new sensor is installed, or when operational policies evolve due to updated regulations or route changes, a fine-tuned model becomes outdated and requires retraining. In safety-critical maritime environments governed by classification society regulations and strict redundancy requirements, such unreliability is unacceptable. Consequently, the architecture adopted in this work employs *structured function calling*, also referred to as tool use, where the LLM invokes deterministic computational functions to retrieve verified data before generating natural language responses. This approach ensures that all numerical values presented to operators originate from the actual data pipeline and from the trained Transformer Autoencoder, not from the language model's parametric knowledge. The distinction is important: the LLM serves as a reasoning and communication layer, while all computation and data retrieval occur outside its inference path.

The system defines six domain-specific tools that collectively provide comprehensive access to the monitoring pipeline, as illustrated in Fig. 3. Each tool is exposed to the LLM as a structured JSON

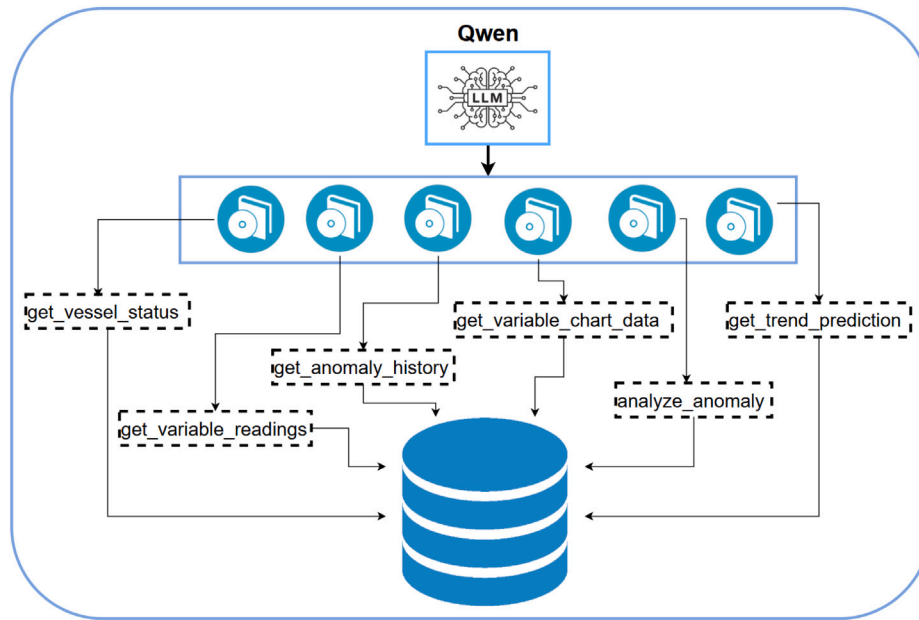


Fig. 3. Tool-calling architecture instantiated in this work. The Qwen3 LLM orchestrates six deterministic tools (`get_vessel_status`, `get_variable_readings`, `get_anomaly_history`, `get_variable_chart_data`, `analyze_anomaly`, and `get_trend_prediction`) that operate on a shared data store populated by the Transformer Autoencoder pipeline.

schema specifying the function name, a natural language description of its purpose, and the expected input parameters with their types and constraints. The first tool, `get_vessel_status`, returns the current anomaly score, severity classification, total power output, vessel speed, GPS position, and the top variables contributing to the anomaly score, providing a comprehensive snapshot of vessel health at any given moment. The second tool, `get_variable_readings`, returns current sensor readings for a specified variable group, such as electrical (bus loads and available capacity), manoeuvre (bow and stern thrusters), propulsion (main drives and engines), ship (draft and speed), or coordinates (GPS position), together with computed aggregate totals where applicable. The third tool, `get_anomaly_history`, scans a configurable lookback window over recent data and returns timestamped anomaly events with their corresponding severity levels, enabling the LLM to reason about temporal patterns and recurring issues. The fourth tool, `get_variable_chart_data`, returns time-series arrays of both actual sensor values and model-reconstructed values for a specified variable, providing the LLM with the information needed to describe trends, deviations, and reconstruction quality. The fifth tool, `analyze_anomaly`, performs detailed analysis of a specific anomaly event by identifying the peak reconstruction error within a time window and determining which sensor features contribute most to the deviation. Finally, the sixth tool, `get_trend_prediction`, fits a linear regression to the recent history of reconstruction errors and returns the estimated time-to-threshold, the slope and intercept of the fitted line, and the coefficient of determination R^2 , enabling the predictive agent to produce prognostic outputs. Each of these tools executes against the live anomaly detector instance, invoking the same trained Transformer Autoencoder and data preprocessing pipeline described in the previous subsection. The tools encapsulate all necessary computation, including data scaling, window extraction, model inference, and metric derivation, and return structured JSON results that the LLM can interpret.

The language model employed in the current implementation is Qwen3 8B (Yang et al., 2025), an open-source model served locally via the Ollama inference framework. This choice avoids dependence on external cloud-based API services, which is particularly relevant for maritime deployments where network connectivity may be intermittent or restricted. When an operator submits a natural language query, the

system initiates a multi-turn interaction protocol. The user message, together with the full conversation history and the tool schema definitions, is sent to the LLM along with a system prompt that explicitly instructs the model to always retrieve data through tool invocations rather than generating numerical values from its parametric memory. The LLM then returns either a direct text response, if it can answer from conversational context alone, or a tool call specifying which function to invoke and with what arguments. In the latter case, the system executes the requested function against the live data pipeline, producing a structured JSON result that is appended to the conversation as a tool-role message. The LLM is then called again with the updated conversation, allowing it to interpret the tool output and decide whether additional tool calls are needed for a complete answer. This iterative loop continues for up to five turns, enabling the model to chain multiple tool invocations when a query requires information from several sources. For example, when an operator asks about an anomaly, the LLM may first invoke `get_vessel_status` to obtain an overview, then call `get_variable_readings` for the electrical group to investigate which bus loads are contributing to the deviation, and finally call `analyze_anomaly` to determine the peak severity and most affected features. Once sufficient information has been gathered, the LLM produces a final natural language response that synthesises the tool outputs into an operationally contextualised explanation.

It is worth noting that this interaction protocol is fundamentally distinct from two other common approaches for integrating language models with external data sources. Retrieval-Augmented Generation (RAG) retrieves unstructured text passages from a document store based on semantic similarity to the user's query and appends them to the LLM's context before generation. In contrast, the approach adopted here invokes typed functions with defined schemas and receives structured computational outputs, not document fragments. Furthermore, while fine-tuning approaches modify the language model's weights to encode domain-specific knowledge, the current architecture leaves the LLM's weights entirely unchanged; only the tool implementations and the underlying anomaly detection model need to be updated when the vessel's configuration or operational characteristics evolve. This separation of concerns between reasoning (LLM) and computation (tools) provides three properties that are particularly important for

safety-critical maritime applications. First, *traceability*: every numerical value that appears in a response can be traced back to a specific tool invocation and ultimately to a specific data window processed by the trained autoencoder, providing the audit trail that is essential for regulatory compliance and post-incident review. Second, *consistency*: given the same vessel state, the tools produce identical outputs regardless of the inherent stochasticity of the language model's generation process; while the LLM may phrase its interpretation differently across runs, the underlying numbers remain deterministic. Third, *adaptability*: when vessel configurations change, for example when a generator is taken offline, a bus-tie configuration is modified, or a new sensor is added to the monitoring system, only the tool implementations need to be updated; the LLM requires no retraining, fine-tuning, or prompt modification beyond adjusting the tool schemas if the function interface changes.

3.3. System description and dataset

To evaluate the proposed approach, operational data obtained from an offshore vessel equipped with a hybrid diesel-electric propulsion system were used. The vessel considered in this study is an Offshore Construction Vessel (OCV) that falls within the class of multipurpose support vessels used in offshore energy and infrastructure operations. These vessels are typically involved in subsea construction activities, offshore equipment installation, maintenance operations, and various engineering activities (Pettersen, 2015). During such operations, the ship's ability to perform precise manoeuvres at low speed and to maintain a stable position is critical. Therefore, OCVs are often equipped with DP systems, electrically powered propulsion systems, and multiple thruster configurations.

The vessel is equipped with a hybrid diesel–electric propulsion and power generation system. It has a total of six diesel generator sets, and the electrical energy produced is distributed to the propulsion systems, manoeuvring systems, and auxiliary ship cargo via the main distribution panel. The main propulsion system consists of two propellers that can operate in either mechanical or electric mode. In mechanical mode, the main diesel engines transmit power directly to the propellers through a reduction gear, whereas in electric mode the propulsion system is powered by electric motors. This hybrid design leads to variations in the electrical system behaviour depending on the vessel's operating mode, particularly during offshore operations. Technical details regarding the ship's electrical generation and distribution system have been previously presented by Osen (2016). The dataset analysed in this study was obtained from real operational measurements collected during the vessel's offshore operations in the North Sea during the late autumn and winter period, through the Integrated Automation System (IAS). The basic characteristics of the vessel and the dataset are given in Table 2.

The dataset includes approximately 91 days of operational records collected at 5-second sampling intervals. It covers a variety of operational variables representing electricity generation and power distribution, including load values and available power reserves for the main distribution busbars, propulsion power, thruster power, ship speed, and geographical location. These variables provide a rich data structure that represents the ship's electrical behaviour across different operating modes.

A preliminary statistical examination of the dataset provides important insights into the vessel's operational profile. The power distribution of the main propulsion system indicates that the values are primarily concentrated in two distinct ranges, corresponding to near-zero power levels and cruising loads of approximately 1500–2000 kW. This suggests that the ship operates within specific modes rather than continuously varying its speed profile. Similarly, an analysis of the speed distribution shows that the ship spends most of its operating time in the 10–12 knots range. Although the design speed is 18 knots, offshore support vessels typically operate at moderate transit speeds

Table 2

Main characteristics of the case-study vessel and dataset (Osen, 2016).

Parameter	Value
Vessel type	Offshore Construction Vessel (OCV)/PSV-AHTS class
Propulsion system	Hybrid diesel-electric architecture
Power generation	Six diesel generator sets supplying the main switchboard
Electrical distribution	Split 690 V main switchboard configuration
Propulsion drive	Electrically driven propulsion motors (dual-mode mechanical/electric)
Thrusters	Bow and stern thrusters for manoeuvring and DP operations
Operational modes	Transit, manoeuvring, and Dynamic Positioning (DP)
Data acquisition system	Integrated Automation System (IAS)
Dataset duration	91 days of operational data
Sampling interval	5 s
Number of monitored variables	16
Measured variables	Bus loads, available power, propulsion power, thruster power, vessel speed, and geographical position

and spend extended periods at low speed or in DP mode during offshore operations. The thruster power levels remain quite low for much of the time series. In offshore operations, particularly in DP mode, thruster systems are mostly used for short periods and for corrective interventions rather than operating at continuously high power. Therefore, the concentration of most thruster measurements at near-zero values can be considered expected operational behaviour. These observations confirm that the dataset represents realistic operating conditions.

It is worth noting that the data described above have been pre-processed. Since they are real-life acquisitions, the original dataset contained outliers and noise that had to be removed for the correct training and testing of the proposed framework. In particular: latitude values that were too small, as well as GPS spikes, were eliminated; duplicate records were removed; and speed data were smoothed through a Moving Average Filter.

3.4. Fault injection

The synthetic anomaly scenarios created in this study were chosen to represent different types of deviations that may occur in the ship's electrical system behaviour. These scenarios are designed to encompass different anomaly patterns, such as reduced available power, load-sharing imbalance, sensor-induced drift, and short-term measurement disturbances. In this way, the ability of the proposed monitoring approach to detect both slowly developing and suddenly emerging deviations can be evaluated.

A crucial step for testing the proposed methodology is to validate it against faulty data. The main practical limitation is that, during the operational life of a machine, most of the acquired data are healthy, and only a small fraction are affected by faults. Consequently, this small amount of data is often not sufficient to properly validate a machine learning or deep learning algorithm (Kougiatsos and Vessa, 2024; Jimenez et al., 2020). For this reason, several authors, such as Rubio et al. (2018), Maione et al. (2025a), have highlighted the importance of using synthetic data. Since the available dataset contained only healthy records, the same approach is followed in the current work.

As discussed by Fang et al. (2024), the degradation law of a machine can follow different mathematical forms, for example exponential or polynomial. Because the focus of the current work is the detection of slow drifts due to time, an additive fault model has been adopted (Maione et al., 2025a):

$$y_{f,i}(t) = y_{h,i}(t) + f_i(t), \quad (7)$$

where $y_{f,i}$ is the faulty value of the i th sensor measurement, $y_{h,i}$ is its healthy value, and $f_i(t)$ is the fault coefficient added to the healthy signal to simulate the degradation pattern.

Among the variables available in the acquired dataset, the bus loads are the most suitable for fault injection. Unlike other variables, such as thruster power, the two bus loads are active throughout the sailing time, making it easier to visualise anomalous behaviour. The following four anomaly scenarios were simulated:

1. a *temporary reduction* of bus available load, representing reduced available power or spinning reserve;
2. a *load-sharing imbalance*, in which the load of Bus 1 increases while the load of Bus 2 decreases (or vice versa);
3. a *slow drift/bias* added over time to mimic sensor drift;
4. *short spikes*. It is worth noting that this is not a real fault but a test case to verify that the sliding-window logic prevents false alarms.

4. Use case and simulation

The development of suitable case studies and simulations is a critical step in validating the proposed framework. As described in Section 3.4, the bus load has been identified as the most suitable variable for fault injection, and four different faults have been injected. These four behaviours can be grouped into two scenario categories:

1. *Temporary anomalies*: these are anomalies that have a limited impact on the asset, namely their occurrence and effects disappear after a certain time. Spikes and temporary reductions can be considered temporary anomalies, although certain differences remain. Spikes are not symptoms of a real fault, as they are caused by noise (e.g., bad weather or sensor inaccuracies), whereas a temporary reduction can be caused by a real fault. The difference between them is the time duration: spikes occur within a small interval; temporary reductions occur over a longer time interval. Consequently, the root causes of the second anomaly must be investigated.
2. *Continuous anomalies*: these faults are characterised by a persistent anomalous behaviour whose severity increases over time. Unlike the previous category, the monitored machines do not return to the healthy condition but enter a faulty condition; if maintenance has not been planned in advance, operators are forced to shut down the engine and plan an unscheduled maintenance action.

To test the capacity of the LLM to correctly interpret both the information provided by the Transformer reconstruction error and the operator's queries, the simulated time-series patterns have been divided into three regions:

1. before the fault injection;
2. after the fault injection and before the fault threshold is reached;
3. after the fault threshold has been reached or after the recovery of normal operations (in the case of temporary anomalies).

These three regions allow the LLM output to be verified in all scenarios of interest. Before the fault injection, the LLM should describe a healthy monitoring situation to the user; in this region, only the monitoring AI agent is operating, evaluating the reconstruction error. In the second region, the predictive AI agent starts computing the linear regression, while the monitoring AI agent continuously provides new reconstruction errors used both for real-time monitoring and for prediction. In the first two regions, the LLM output is similar for all four injected faults, with the only difference being the predicted values. Finally, in the third region, the LLM outputs differ depending on the type of fault injected: in the case of temporary anomalies, the LLM should recognise that the propulsion plant has returned to healthy

behaviour; in the case of continuous anomalies, the system enters a faulty operating condition that would force a shutdown. The goal of the predictive AI agent is precisely to avoid this scenario by providing reliable predictions that allow the required maintenance action to be planned with suitable lead time.

5. Results and discussion

For testing the proposed approach, the first step is to inject the faults following the procedure described in Section 3. In the following figures, the bus load is expressed in kilowatts (kW). Fig. 4 gives an overview of the simulated slow drift on the two bus loads. At the beginning, the acquired sensor measurements show healthy behaviour; after a certain point, a drift is observed.

Fig. 5 shows the load imbalance case. Bus load 1 is affected by an anomalous decrease, while Bus load 2 experiences a corresponding increase of its expected values.

Fig. 6 illustrates the temporary reduction fault, and Fig. 7 shows a temporary spike. As already noted, the latter is not a real fault; spikes are due to bad weather or noise, and the purpose is to check that the sliding-window logic does not produce a false alarm.

Once the injected faults have been described, the next step is to train the Transformer Autoencoder. The model was trained on healthy operational data, split temporally into training (70%), validation (15%), and test (15%) sets. Sliding windows of 120 time steps were extracted with a stride of 12 samples. Training was performed using the AdamW optimiser for 50 epochs with a batch size of 512. The model achieved the best validation loss at epoch 22, with a training loss of 0.0176 and a validation loss of 0.0360 (MSE). The total number of trainable parameters is 243 280. Fig. 8 shows the training and validation loss curves over all 50 epochs, with the best checkpoint indicated by the dashed vertical line.

The anomaly detection threshold was computed by evaluating the trained model on the full healthy dataset. Reconstruction errors were smoothed using a sliding window average of 50 samples. The threshold was set at 25% above the maximum smoothed healthy error, yielding $\epsilon_{\text{threshold}} \approx 0.408$. The healthy baseline mean reconstruction error was 0.022 with a standard deviation of 0.042, confirming a good separation between the healthy distribution and the threshold.

The 25% margin above the maximum smoothed healthy error is a deliberately simple and interpretable choice. In a safety-critical, adoption-reluctant domain, a transparent max-plus-margin rule has the advantage of being fully explainable to operators and of requiring no fault data, which is consistent with the traceability objective of the framework; the margin provides a guard band against the residual noise and intrinsic reconstruction error that persist even on healthy data. To verify that the specific margin value does not drive the reported results, a sensitivity analysis was performed by recomputing the threshold at 15% and 40% and re-evaluating all four fault scenarios; the outcomes are reported in Table 3. Across this range the threshold varies between approximately 0.375 and 0.457, which remains far above the healthy baseline mean (0.022) and far below the fault reconstruction-error peaks (up to 0.99). Consequently, the detection outcome, whether each fault is ultimately flagged, is unchanged across all three margins; the only effect is a modest shift in the prognostic lead time, as an earlier or later intersection between the extrapolated error trend and the threshold. This confirms that the framework is robust to the precise choice of margin within a reasonable range, while more principled thresholding strategies (e.g., extreme-value or distribution-based statistical limits, or adaptive thresholds conditioned on operational mode) are identified as future work in Section 6.

The Graphical User Interface (GUI) was developed using Gradio 6.5, an open-source Python framework for building interactive web applications, together with Python 3.12. The Transformer Autoencoder was implemented in PyTorch 2.0, and LLM inference is served locally

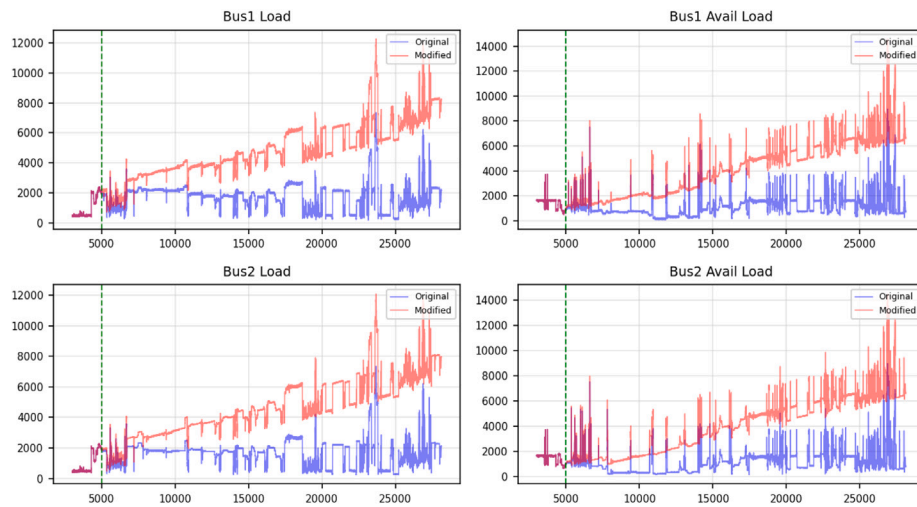


Fig. 4. Bus load affected by slow drift. The original (healthy) signal is shown in blue and the modified (faulty) signal in red; the vertical dashed line indicates the fault injection instant.

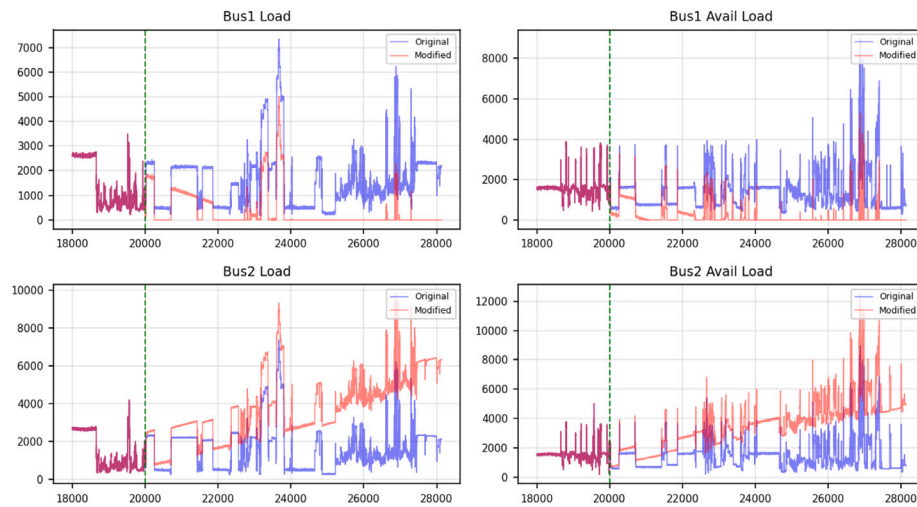


Fig. 5. Bus load affected by load imbalance; the opposite trends on Bus 1 and Bus 2 are clearly visible after the injection point.

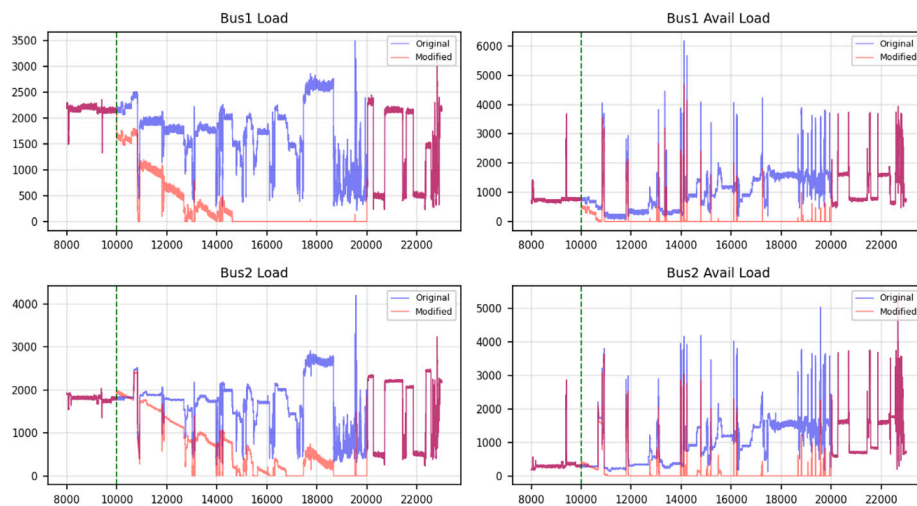


Fig. 6. Bus load affected by a temporary reduction.

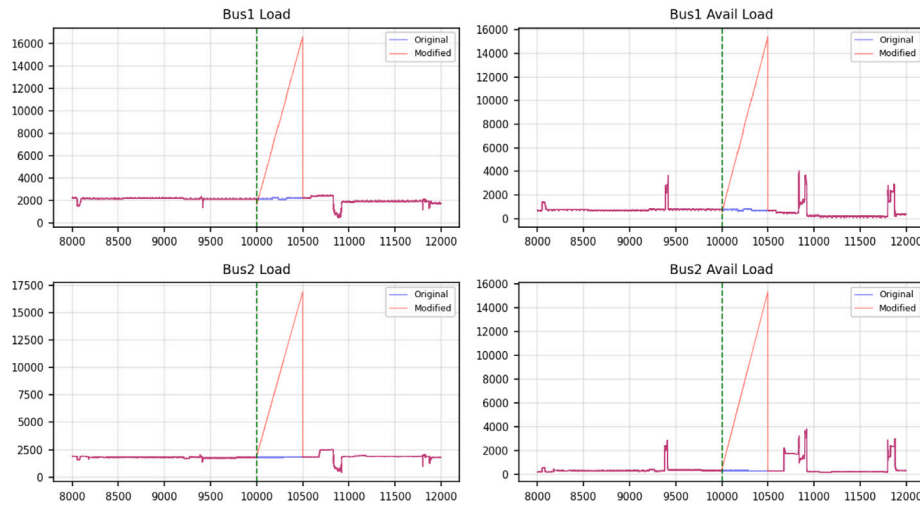


Fig. 7. Bus load affected by a spike. Spikes are injected as a robustness test rather than as a real fault.

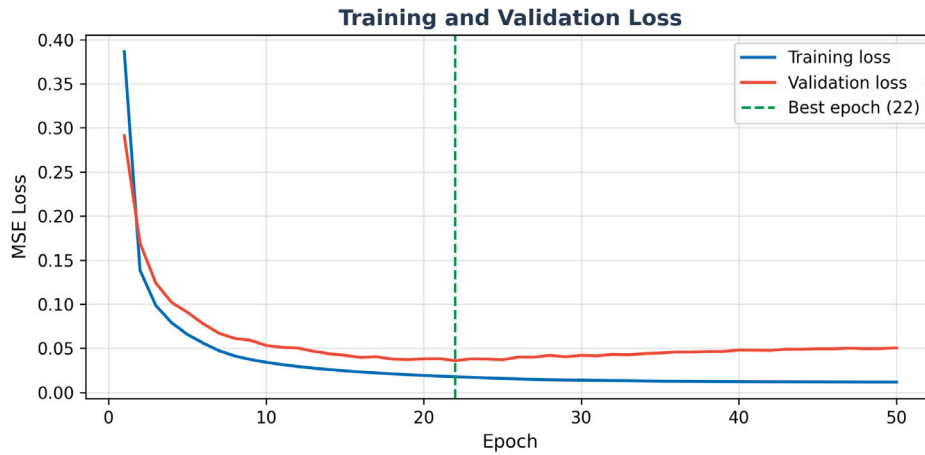


Fig. 8. Training and validation loss (MSE) over 50 epochs. The dashed line indicates the best checkpoint at epoch 22, selected based on the lowest validation loss.

Table 3

Sensitivity of the results to the threshold margin. Detection outcome is unaffected across the tested range; only the prognostic lead time shifts.

Margin	$e_{\text{threshold}}$	All 4 faults detected?	Lead-time change vs. 25%
15%	≈ 0.375	Yes	Earlier crossing (larger lead time)
25% (used)	≈ 0.408	Yes	Baseline
40%	≈ 0.457	Yes	Later crossing (smaller lead time)

through Ollama, avoiding dependence on external cloud-based API services.

Because the intended application is onboard maritime deployment, where computational resources and network connectivity are constrained, the computational cost of the two pipeline components was characterised, and their profiles differ markedly. The Transformer Autoencoder is extremely lightweight: with only 243 280 trainable parameters, it occupies approximately 1 MB of memory and processes a single 120×16 window in a few milliseconds at inference, running comfortably on standard onboard CPU hardware. The dominant cost is the language-model reasoning layer: Qwen3 8B, served locally through Ollama, produces a complete tool-calling response of up to five turns within a latency suited to interactive use. Crucially, the language model is invoked only in response to operator queries or confirmed anomaly events, rather than continuously at the 5-second sampling rate, so its cost does not affect the real-time detection loop. The deliberate choice

of a locally-served open model rather than a cloud API is motivated precisely by the intermittent connectivity typical of maritime operations. For the most resource-constrained vessels, model quantisation or smaller language models represent a natural optimisation, identified as future work (Section 6).

The next subsections describe the interactions between the human operator and the LLM-based Decision Support system for each of the case studies identified in Section 4.

5.1. Slow drift and load imbalance

The first fault considered is the slow drift. Before the fault injection, an operator requesting health information for the monitored propulsion plant receives the response shown in Fig. 9. At the top of the GUI, the system shows the reconstruction error computed in real time (centre, upper side) and the acquired values (lower side); the blue curves represent the healthy sensor measurements and the red curves the degradation pattern. In the error chart, the red dashed line is the threshold that, when crossed, identifies a detected fault. The violet marker indicates the point in time at which the user asks a question to the system. On the right-hand side, the user chats with the LLM. Both AI agents (monitoring and predictive) correctly report no anomaly in the system, and this information is communicated to the user. The exchange is as follows:

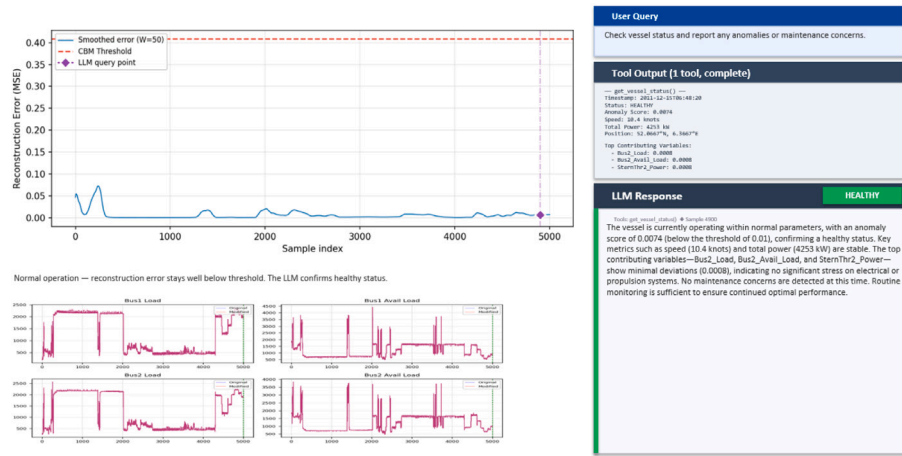


Fig. 9. GUI overview before the injection of the slow drift. The reconstruction error remains well below the threshold and both AI agents report a healthy status.

- *User query:* Check vessel status and report any anomalies or maintenance concerns.
- *LLM response:* The vessel is currently operating within normal parameters, with an anomaly score of 0.0074 (well below the threshold of 0.408), confirming a healthy status. Key metrics such as speed (10.4 knots) and total power (4253 kW) are stable. The top contributing variables, such as Bus2_Load, Bus2_Avail_Load, and SternThr2_Power, show minimal deviations (0.0008), indicating no significant stress on electrical or propulsion systems. No maintenance concerns are detected at this time. Routine monitoring is sufficient to ensure continued optimal performance.

After the fault is injected, the GUI updates its interface and shows the linear regression fitted to the reconstruction error, predicting the expected time of fault threshold crossing (Fig. 10). In this figure, the solid orange line is the linear regression computed on the evaluated error measurements, while the dashed orange line represents the predicted errors. The user asks for information and receives the following answer:

- *User query:* Reconstruction error is rising. Analyse electrical readings and predict the trend.
- *LLM response:* The vessel's anomaly score (0.2649) remains below the threshold (0.408), confirming a healthy status. However, the reconstruction error is trending upward with a slope of 1.23×10^{-4} and an R-squared of 0.845, indicating a statistically significant rise. Electrical readings show Bus1_Load (6211 kW) and Bus2_Load (6063 kW) at 100% capacity, with available load margins (5104 kW and 5002 kW) suggesting potential overload risks if demand increases. Proactive monitoring of electrical distribution systems is recommended to mitigate risks before the anomaly score reaches the threshold in approximately 96.8 min. The system is currently healthy but requires attention to prevent escalating issues.

The monitoring agent correctly reports that no anomaly has been detected yet, while the predictive agent highlights the risk of an imminent fault. From an operational perspective, this behaviour can be associated with gradual sensor drift or with equipment degradation developing over time, which is typically difficult to detect using conventional monitoring approaches.

Finally, the reconstruction error reaches the threshold, and the GUI highlights this information, as shown in Fig. 11. The user-LLM exchange is the following:

- *User query:* Anomaly threshold crossed. Analyse the anomaly and identify contributing variables.

- *LLM response:* The vessel's anomaly score of 0.5129 exceeds the threshold, indicating a WARNING status. The peak anomaly score of 0.9937 highlights severe deviations, primarily driven by Bus1_Load (4574.5 kW) and Bus1_Avail_Load (4449.5 kW), suggesting potential overloading or mismatch in electrical distribution. Bus2_Load (4399.5 kW) also contributes, indicating uneven load distribution across buses. The electrical system is the primary affected area, with risks of overheating or component failure. Immediate maintenance is recommended to investigate load balancing and resolve discrepancies. The vessel's electrical systems require urgent inspection to prevent further degradation.

The load imbalance case yields results that are qualitatively similar to the slow drift, since the underlying physical behaviour is comparable. Fig. 12 shows that the system recognises different anomaly behaviours for Bus 1 and Bus 2, and that the predictive agent starts evaluating the linear regression. Fig. 13 then shows the GUI output once the fault has occurred. In real operations, this type of imbalance may indicate non-ideal load sharing between generators and can be used as an indicator to assess the performance of the Power Management System (PMS).

5.2. Temporary reduction and spikes

These case studies are interesting because they allow an assessment of how the developed system manages temporary anomalous behaviours. Starting with the temporary reduction, and omitting the initial interaction when the system is healthy (which is similar to the previous case studies), once the bus load reduction is detected, both AI agents recognise the anomalous behaviour and communicate the information to the user. As shown in Fig. 14, the predictive agent starts evaluating the linear regression to predict the potential fault occurrence:

- *User query:* Reconstruction error is rising. Analyse electrical readings and predict the trend.
- *LLM response:* The vessel's current status is HEALTHY with an anomaly score of 0.0987, below the threshold of 0.408. However, the reconstruction error is RISING with a slope of 4.05×10^{-5} and an R-squared of 0.47, indicating a gradual upward trend. Electrical readings show Bus1_Load and Bus1_Avail_Load at 0.00 kW, while Bus2_Load and Bus2_Avail_Load account for 172.10 kW, matching the total power output. The trend suggests potential system instability, though no immediate faults are detected. Proactive monitoring of Bus1 load metrics is recommended to prevent reaching the threshold in approximately 636.8 min. The system remains healthy but requires vigilance.

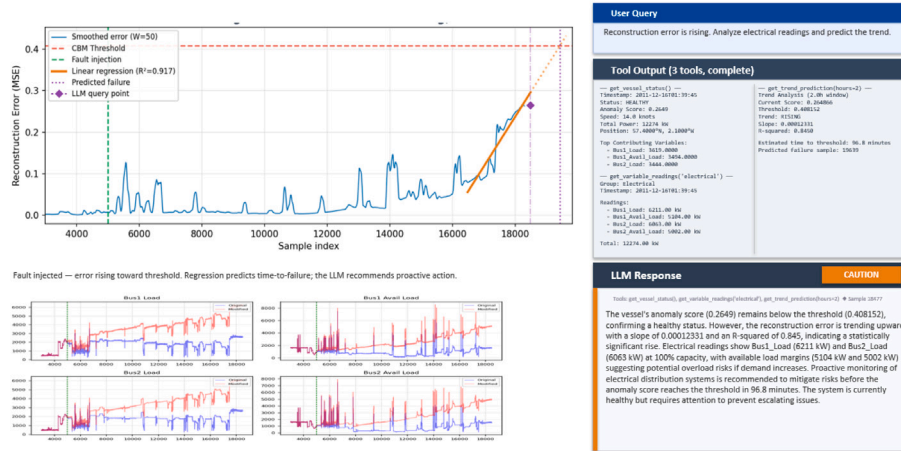


Fig. 10. GUI overview after the injection of the slow drift. The predictive agent has fitted a linear regression to the rising reconstruction error and provides a time-to-threshold estimate, while the monitoring agent still reports a healthy status.

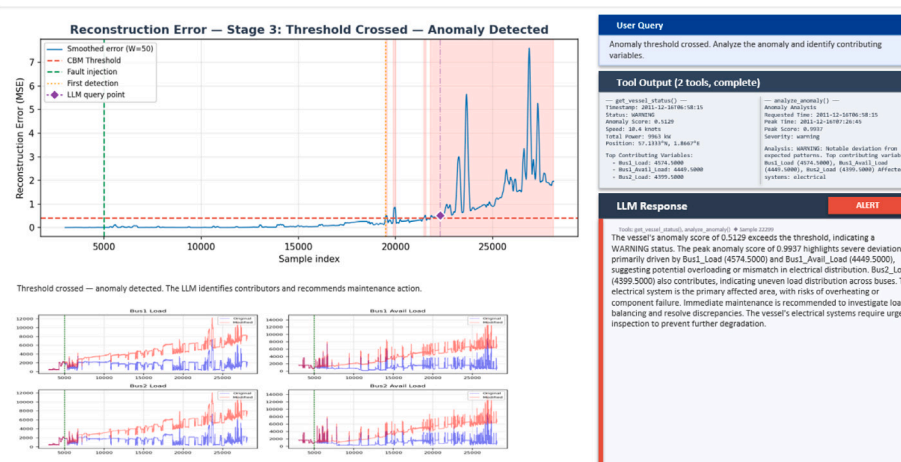


Fig. 11. GUI overview when the fault threshold is crossed (slow drift case). The LLM automatically identifies the main contributing variables.



Fig. 12. GUI overview when the load imbalance starts. Bus 1 and Bus 2 display opposite trends, and the predictive agent begins fitting the linear regression.

From an operational perspective, this behaviour can be associated with temporary limitations in generator capacity or with reduced spinning reserve, which may indicate a critical condition for the PMS.

The bus load then returns to its expected behaviour, and both the monitoring and the predictive agents communicate this information

to the user. In Fig. 15, it is clear that the predictive agent no longer performs the linear interpolation of the error, because the error trend has reverted to nominal.

In the presence of spikes, the system exhibits a similar behaviour: an increasing reconstruction error is detected (Fig. 16), after which the

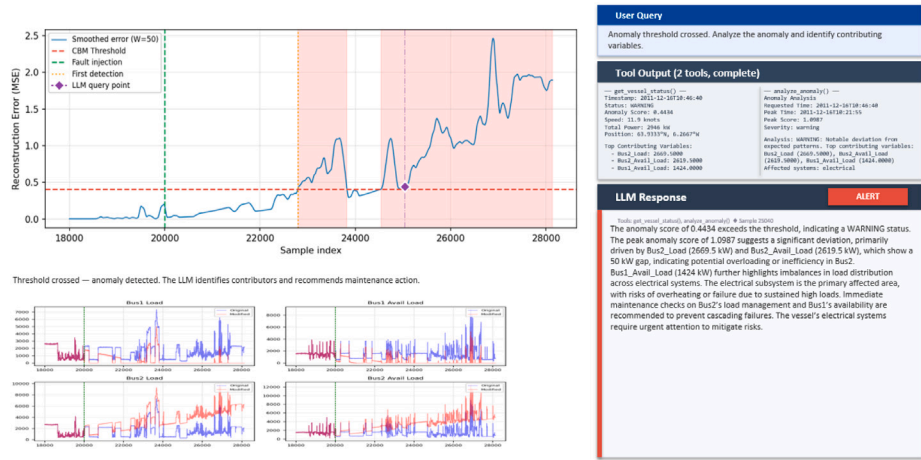


Fig. 13. GUI overview when the load imbalance evolves into a full fault.

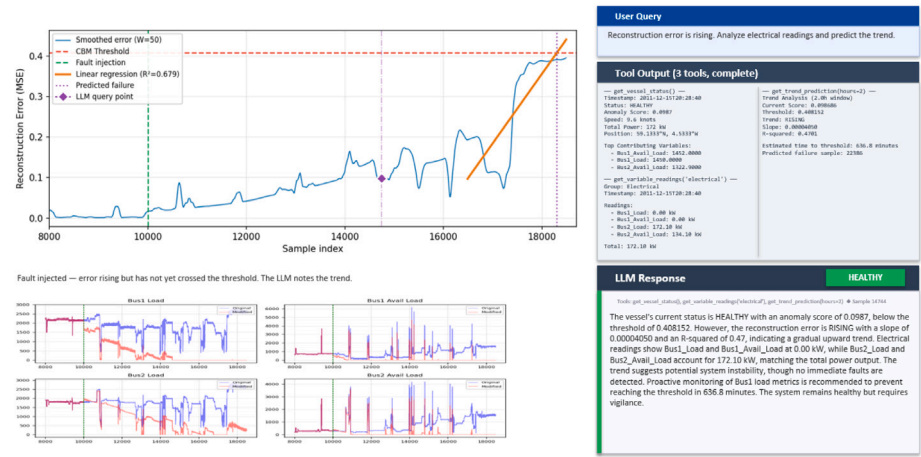


Fig. 14. GUI overview in the case of a temporary reduction. The predictive agent reports a long time-to-threshold with a moderate R^2 , reflecting the fact that the anomaly is not permanent.



Fig. 15. GUI overview once the bus load reduction is no longer present. The predictive agent has deactivated the linear regression as the error trend has returned to healthy levels.

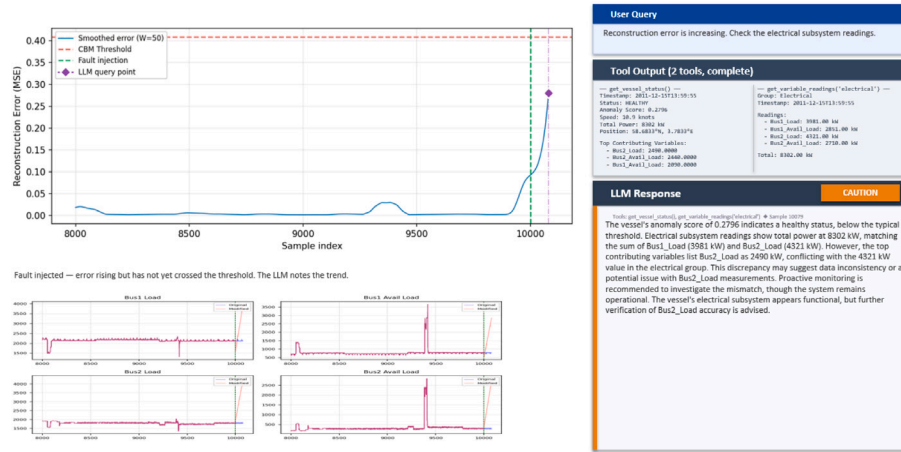


Fig. 16. GUI overview when a spike is detected. The smoothed error briefly rises but does not cross the threshold persistently.

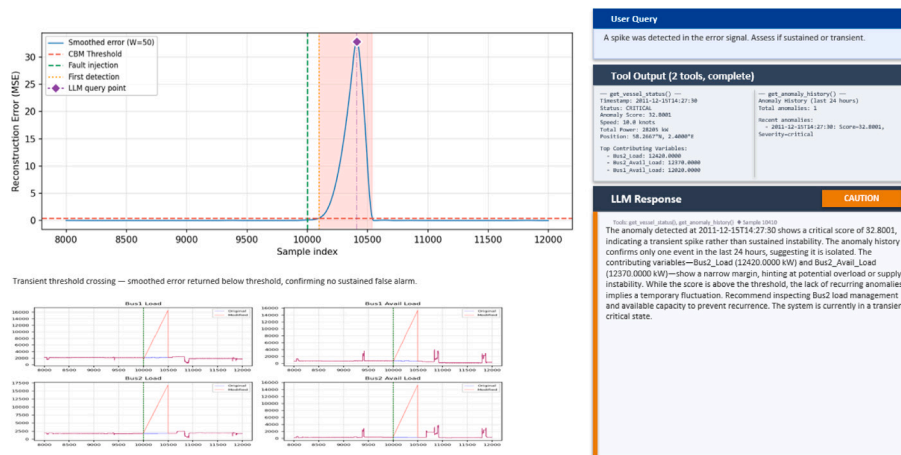


Fig. 17. GUI overview once the spike is no longer present. The system correctly treats the event as transient and does not raise a sustained alarm.

situation quickly returns to a healthy one (Fig. 17). The main difference with respect to the temporary reduction is that spikes last for a shorter time. During a temporary reduction, the anomalous behaviour lasts long enough to activate the predictive agent; in the case of spikes, the anomaly is so short that the 50-sample sliding window average attenuates its effect and prevents the prediction of a false alarm. Such short and non-persistent spikes may originate from transient operational disturbances (e.g., environmental effects or sensor noise) and should not necessarily trigger alarms on their own. From an operational point of view, the predominance of low thruster power levels observed in the dataset is consistent with DP operations, where thrusters are mainly used for short corrective actions rather than for continuous high-load operation.

5.3. Summary and quantitative performance

Taking into account all four simulated faults, the results can be summarised as follows:

- **Sustained faults (slow drift, load imbalance):** detected reliably, with linear regression providing time-to-failure predictions and coefficients of determination of $R^2 = 0.84$ and $R^2 = 0.90$, respectively.
- **Temporary reduction:** detected during the fault period; the reconstruction error returns to normal values after the fault ends, confirming that the monitoring agent is sensitive to operational recovery.

- **Spikes:** the transient is detected locally but does not generate a sustained false alarm; the 50-sample sliding window average correctly suppresses brief disturbances.
- **LLM layer:** contextualised natural language interpretations are provided at each stage, grounded in verified tool outputs and consistent with the displayed quantitative indicators.

Table 4 summarises the performance of the predictive and monitoring agents across the four scenarios.

Because the faults are synthetically injected with known onset and recovery instants, detection performance is reported here through operationally meaningful quantities, detection delay and prognostic lead time (Table 4), the separation between the healthy reconstruction-error baseline and the threshold, and the demonstrated suppression of transient false alarms, rather than through window-level classification metrics alone. Standard classification metrics such as Precision, Recall, and F1-score are most informative when evaluated against real, independently labelled failure events; their computation on self-injected anomalies would largely reflect the detectability of the designed perturbations. A full classification-metric evaluation is therefore deferred to the planned validation on recorded real failures (Section 6).

All simulated faults were correctly detected. In Table 4, the *detection delay* refers to the time interval between the initial injected fault condition (which is still considered healthy by the monitoring agent) and the crossing of the fault threshold. Regarding the predicted time of the fault, the linear regression anticipates the threshold crossing by some time ahead. Even though the error is typically on the order of

Table 4

Performance of the predictive and monitoring AI agents on the four simulated fault scenarios.

Fault type	Detected?	Detection delay	Prognostic lead time
Slow drift	Yes	1206 min (14 478 samples)	~97 min ahead ($R^2 = 0.84$)
Load imbalance	Yes	233 min (2794 samples)	~126 min ahead ($R^2 = 0.90$)
Temporary reduction	Yes	791 min (9491 samples)	~637 min ahead ($R^2 = 0.47$)
Spikes	Yes	8 min (96 samples)	N/A (transient)

one or two hours, it is considered acceptable given that the dataset covers 91 sailing days. In practical terms, maintenance actions should be planned several days before the expected fault occurrence, so errors on the order of one hour do not affect the safety of the ship, cargo, or personnel. Similarly, the 637-minute lead time obtained for the temporary reduction (corresponding to approximately 10 h) is still acceptable. The relatively low R^2 obtained for the temporary reduction case reflects the non-monotonic nature of the underlying signal and is consistent with the fact that this is a self-recovering anomaly: the predictive agent conservatively extrapolates a future threshold crossing that does not actually occur, but the monitoring agent subsequently confirms the return to a healthy state and the linear regression is deactivated.

It should be emphasised that in all case studies the LLM correctly contextualised the outputs of the monitoring and predictive agents, translating anomaly scores, R^2 values, and contributing variables into operator-oriented language without generating any numerical values from its parametric memory. This behaviour is a direct consequence of the structured tool-calling design: the numbers appearing in each LLM response can be traced back to a specific tool invocation, providing the traceability required for safety-critical decision support.

The robustness and generalisation behaviour of the framework can be characterised along several dimensions, drawing together evidence distributed across the preceding results. First, robustness to transient disturbances is demonstrated by the spike scenario, in which the 50-sample sliding-window average suppresses short-lived noise without producing a sustained false alarm. Second, robustness to the thresholding parameter is established by the sensitivity analysis of Table 3, which shows that the detection outcome is invariant to margin choices between 15% and 40%. Third, temporal generalisation is inherent in the evaluation protocol: the training, validation, and test splits are chronological (70/15/15), so the reported reconstruction performance is obtained on future operational data that the model never observed during training. Fourth, the detector is trained on healthy data spanning the vessel's principal operating modes, transit, manoeuvring, and dynamic positioning, and learns a single normal-behaviour manifold across these modes without mode-specific tuning, indicating generalisation across the operational envelope represented in the dataset. Finally, the reasoning layer is robust by construction: because all numerical outputs are produced by deterministic tools rather than by the language model, identical vessel states yield identical quantitative outputs regardless of the stochasticity of language generation. Generalisation across different vessels and power-system configurations, by contrast, cannot be established from a single-vessel dataset and is identified, together with real-fault validation, as a priority for future work (Section 6).

It should also be acknowledged that the present validation relies on synthetically injected faults rather than on recorded real failure events. This is a direct consequence of the operational reality of the case-study vessel: like most safety-critical marine assets, it is designed with high reliability, built-in redundancy, and strict safety margins, so that genuine failure events are rare and were absent from the 91-day acquisition. Acquiring real faulty data is inherently difficult; acquiring even real healthy data from operational vessels is itself non-trivial, owing to industrial confidentiality and, in related domains, military restrictions. The approach adopted here mitigates these constraints by injecting faults onto real healthy operational measurements, so that

the underlying baseline retains authentic sensor noise, operational-mode transitions, and inter-variable correlations; this increases the realism and reliability of the simulated anomalies compared with fully synthetic or generic benchmark datasets. Importantly, the injected faults are used exclusively to test the detection and prognostic logic; the Transformer Autoencoder is trained solely on healthy data and never observes the synthetic anomalies, which avoids any circularity between the injected fault shapes and the learned model. The four scenarios were, moreover, designed to reproduce physically meaningful deviations of the vessel's electrical system, such as reduced available power and spinning reserve, load-sharing imbalance, sensor drift, and transient disturbances, using an additive fault model consistent with the condition-monitoring literature. Synthetic anomalies nevertheless cannot fully capture the complexity of real degradation, and validation on recorded failure events remains an essential next step, as discussed in Section 6.

5.4. Safety, governance, and verification considerations

Because the target domain is safety-critical, it is important to make explicit the safety-relevant properties of the proposed architecture, the failure modes it may exhibit, and the boundaries of human responsibility. Several safety-by-design properties are intrinsic to the framework and have been referenced throughout the paper: all numerical outputs are produced by deterministic tools rather than by the language model, so every reported value is traceable to a specific computation and data window, providing the audit trail required for post-incident review; the language model operates as a bounded reasoning layer under a fixed tool schema and a limited interaction loop, so its autonomy is structured rather than open-ended; and the detection stage incorporates a conservative threshold margin and sliding-window smoothing to limit spurious alarms. These properties are consolidated here into an explicit treatment rather than being left implicit. A structured, failure-mode-oriented analysis of the framework is summarised in Table 5, which lists the principal failure modes, their likely causes, the mitigations already embedded in the architecture, and the residual risks that motivate further work. The analysis distinguishes failures of the detection layer (missed detections and false alarms), of the reasoning layer (ungrounded interpretation and incorrect tool use), of the data interface (corrupted or missing sensor input), and of the prognostic layer (over-confident linear extrapolation). In several cases the mitigation has been demonstrated empirically in Section 5: the sliding-window average suppressed transient spikes without raising a sustained alarm, and the monitoring agent correctly deactivated an over-extrapolated prognosis once the temporary reduction recovered. Regarding human responsibility, the framework is explicitly a decision-support system and not an autonomous decision-maker. The language model retrieves, computes, interprets, and communicates, but the authority to schedule maintenance, alter operational modes, or take safety actions remains with the certified crew and shore-side operators, who retain full accountability. In the taxonomy of human oversight, the system operates human-on-the-loop: the agents propose grounded, explainable assessments, while a qualified human remains the final decision authority, consistent with the expectations of classification-society guidance for autonomous and remotely supported vessels. This division of responsibility is a deliberate design constraint rather than an implementation detail. Because human operators are the final decision authority, the usability and trust characteristics of the operator interface are themselves safety-relevant.

Table 5

Structured failure-mode analysis of the proposed framework: principal failure modes, causes, embedded mitigations, and residual risks.

Failure mode	Cause	Embedded mitigation	Residual risk
Missed detection (false negative)	Novel deviation outside the learned normal manifold, or subtle sub-threshold change	Per-feature error decomposition surfaces partial deviations; human-on-the-loop oversight	Genuinely novel failure modes may evade detection; motivates real-fault validation
False alarm (false positive)	Transient disturbance, sensor noise, or benign unmodelled mode	25% threshold margin; 50-sample sliding-window average (demonstrated on spikes)	Rare benign modes absent from training may trigger alarms
Ungrounded interpretation	Language-model parametric inference	Structured tool-calling; the LLM never emits numerical values from memory; full traceability	Correctly retrieved values may still be phrased imprecisely; human review required
Incorrect tool use	Reasoning error in tool selection or arguments	Typed JSON schemas; bounded five-turn loop; deterministic tools	An incorrect call may yield an incomplete answer; operator can re-query
Corrupted or missing sensor input	Sensor fault, dropout, or spikes	Preprocessing (outlier removal, moving-average smoothing)	Undetected sensor faults propagate; a dedicated sensor-validation layer is future work
Operational distribution shift	Genset offline, new sensor, or policy change	Tools and detector updated without LLM retraining (adaptability)	Detector itself requires retraining on new-configuration healthy data
Overconfident prognosis	Non-linearity of real degradation vs. linear extrapolation	Monitoring agent confirms or deactivates prediction (demonstrated on temporary reduction)	No confidence interval; probabilistic prognostics identified as future work

A dedicated human-factors evaluation, measuring operator trust, interpretability, cognitive workload, and decision quality when using the system, was outside the scope of this feasibility study; it is planned as a structured follow-up, described in Section 6. A complete, formal safety case and an independent verification-and-validation programme against the relevant maritime assurance standards are beyond the scope of this feasibility study and are identified as essential future work in Section 6.

6. Conclusion

This work contributes to the field of data-driven condition-based maintenance of marine vessels by proposing an LLM-based Agentic AI architecture for autonomous condition monitoring and predictive maintenance. The architecture is built around a Transformer Autoencoder trained to reconstruct healthy sensor measurements acquired from the real propulsion plant of an Offshore Construction Vessel. Two specialised AI agents, a monitoring agent and a predictive agent, operate on top of this reconstruction layer: the first evaluates the reconstruction error as an indicator of the current state of health, while the second fits a linear regression to the error history to predict future threshold crossings. A Qwen3 8B LLM, orchestrating six domain-specific tools through structured function calls, acts as the reasoning and communication layer between the agents and the operator. The framework was validated on 91 days of real operational data under four representative fault scenarios. All four faults were correctly detected: the sustained faults (slow drift and load imbalance) were captured with high-confidence linear extrapolations (R^2 of 0.84 and 0.90, respectively), providing prognostic lead times of approximately 97 and 126 min; the temporary reduction was detected both during and after the event, with the predictive agent automatically deactivating when the error returned to nominal values; and the injected spikes did not generate sustained false alarms, confirming the effectiveness of the 50-sample sliding window average. At each stage, the LLM produced contextualised, grounded explanations consistent with the tool outputs, without generating any numerical value from its parametric memory. Beyond the numerical performance, the main contribution of the work lies in demonstrating the feasibility of integrating deep anomaly detection, prognostic reasoning, and tool-grounded language interaction within a single autonomous framework tailored to the constraints of

safety-critical maritime systems. The separation of concerns between reasoning (LLM) and computation (tools) provides three properties that are particularly relevant for marine applications: traceability of every reported number to a specific computation; consistency of the underlying quantitative outputs across different phrasings of the same query; and adaptability to changing vessel configurations without re-training the language model. Several directions are envisaged for future research. First, the framework will be deployed in more realistic operational settings, moving from synthetic fault injection to extended trials on additional vessels with a richer variety of subsystems, following an approach similar to [Hodne et al. \(2024\)](#). In particular, it will be important to understand how onboard and onshore operators interact with the system. Even though the long-term goal is the development of fully autonomous ships, in the medium term it is essential to support crew members through this digital transition by developing trustworthy systems that enhance the synergy between human operators and AI agents. To this end, the research group will involve nautical students and experts who, through a ship simulator, will interact with the LLM-based support system in a controlled human-factors evaluation. This study will assess operator trust, the interpretability and perceived usefulness of the explanations, cognitive workload, and the quality of maintenance decisions taken with and without agentic support, using established usability and trust instruments together with structured feedback, in order to refine the system's design and outputs. Second, the multi-agent structure outlined in Section 2 will be extended beyond the current two-agent instantiation, introducing dedicated agents for propulsion diagnostics, fuel efficiency, and regulatory compliance, with communication orchestrated through a shared vector memory. Third, the prognostic layer, currently based on linear regression, will be replaced by more sophisticated time-series models (e.g., Bayesian structural time-series or probabilistic Transformers) in order to provide confidence intervals around the predicted time-to-threshold. Fourth, a controlled quantitative benchmark of the reconstruction-based detector against established unsupervised baselines, in particular an LSTM Autoencoder and a PCA reconstruction model, will be carried out under identical training data, window size, and thresholding conditions, reporting standard detection metrics, in order to characterise the accuracy-complexity trade-off of the attention-based detector within the proposed framework. A further essential direction concerns formal assurance. The present study establishes safety-by-design properties

and a structured failure-mode analysis (Section 5), but a complete AI safety case, together with an independent verification-and-validation programme assessed against the emerging classification-society and international guidance for autonomous and AI-assisted vessels, remains to be developed before operational deployment. Establishing this assurance framework, including quantitative reliability targets for the detection and prognostic layers, is a prerequisite for certification and constitutes a central objective of subsequent work. Together, these extensions are expected to move the proposed framework closer to the Shipping 4.0 paradigm and to consolidate Agentic AI as a practical enabler of safer, more efficient, and more sustainable maritime operations.

CRedit authorship contribution statement

Francesco Maione: Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Conceptualization. **Seher Suen-dam Arici:** Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization. **Agus Hasan:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Formal analysis, Conceptualization. **Modestas Sukarevicius:** Writing – original draft, Visualization, Validation, Methodology, Conceptualization. **Emre Akyuz:** Writing – original draft, Supervision, Methodology, Investigation, Conceptualization. **Anniken Karlsen:** Writing – original draft, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Engine data (Original data) (Github)

References

- Alfarizi, Muhammad Gibran, Tajiani, Bahareh, Vatn, Jørn, Yin, Shen, 2022. Optimized random forest model for remaining useful life prediction of experimental bearings. *IEEE Trans. Ind. Inform.* 19 (6), 7771–7779.
- Alkayyal, Majd, Malberg, Simon, Groh, Georg, 2025. An LLM-based decision support system for strategic decision-making. In: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, pp. 460–464.
- Bampoula, X., Nikolakis, N., Alexopoulos, K., 2024. Condition monitoring and predictive maintenance of assets in manufacturing using LSTM-autoencoders and transformer encoders. *Sensors* 24 (10).
- Chae, Chong-Ju, 2025. The evolution of maritime technology development: a dynamic positioning system perspective of maritime autonomous surface ship. *WMU J. Marit. Aff.* 24 (1), 99–127.
- Chen, Xuefeng, Lei, Yaguo, Li, Yanfu, Parkinson, Simon, Li, Xiang, Liu, Jinxin, Lu, Fan, Wang, Huan, Wang, Zisheng, Yang, Bin, et al., 2025. Large models for machine monitoring and fault diagnostics: Opportunities, challenges, and future direction. *J. Dyn. Monit. Diagn.* 4 (2), 76–90.
- Cui, Benben, Wang, Chen, Wu, Yun, Zhu, Can, Lin, Jianmin, Yang, Tingjie, 2024. Intelligent monitoring of marine vessel dynamics based on data mining. *Ocean Eng.* 313, 119387.
- Di Bernardo, R., Di Cecca, B., Coppola, T., Spazzafumo, G., Speranza, D., 2025. Preliminary design of a 75 m Mega Yacht with diesel-Electric hybrid propulsion powered with hydrogen FCs. *Int. J. Hydrog. Energy* 137, 917–924.
- Ellefsen, A.L., Bjørlykhaug, E., Aesøy, V., Zhang, H., 2019. An unsupervised reconstruction-based fault detection algorithm for maritime components. *IEEE Access*.
- Fang, X., Blesa, J., Puig, V., 2024. Fault prognosis approach using data-driven structurally generated residuals. In: *32nd Mediterranean Conference on Control and Automation (MED)*, 11–14 June, Chania, Greece.
- Glorot, Xavier, Bengio, Yoshua, 2010. Understanding the difficulty of training deep feedforward neural networks. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. In: *Proceedings of Machine Learning Research*, vol. 9, PMLR, pp. 249–256.
- Han, P., Ellefsen, A.L., Li, G., Aesøy, V., Zhang, H., 2021a. Fault prognostics using LSTM networks: Application to marine diesel engine. *IEEE Sens. J.* 21 (22).
- Han, P., Ellefsen, A.L., Li, G., Holmeset, F.T., Zhang, H., 2021b. Fault detection with LSTM-based variational autoencoder for maritime components. *IEEE Sens. J.* 21 (19).
- Heo, Sangwoo, Son, Sungwook, Park, Hyunwoo, 2025. HaluCheck: Explainable and verifiable automation for detecting hallucinations in LLM responses. *Expert Syst. Appl.* 272, 126712.
- Hodne, P., Skåden, O.K., Alsos, O.A., Madsen, A., Porathe, T., 2024. Conversational user interfaces for maritime autonomous surface ships. *Ocean Eng.* 310, 118641.
- Huang, Lei, Yu, Weijiang, Ma, Weitao, Zhong, Weihong, Feng, Zhangyin, Wang, Haotian, Chen, Qianglong, Peng, Weihua, Feng, Xiaocheng, Qin, Bing, Liu, Ting, 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.* 43 (2), 1–55.
- Jimenez, V.J., Bouhmal, N., Gausdal, A.H., 2020. Developing a predictive maintenance model for vessel machinery. *J. Ocean Eng. Sci.* 5.
- Kolodziejski, Marcin, Michalska-Pozoga, Iwona, 2023. Battery energy storage systems in ships' hybrid/electric propulsion systems. *Energies* 16 (3), 1122.
- Kougiatsos, N., Vessa, R., 2024. A distributed cyber-physical framework for sensor fault diagnosis of marine internal combustion engines. *IEEE Trans. Control Syst. Technol.* 32 (5), 1718–1729.
- Loshchilov, Ilya, Hutter, Frank, 2019. Decoupled weight decay regularization.
- Maione, F., Lino, P., Giannino, G., Maione, G., 2025a. A new deep learning strategy to predict time-series of marine diesel engines. *IEEE Trans. Ind. Appl.*
- Maione, F., Lino, P., Maione, G., Giannino, G., 2025b. A machine learning method to early detect catastrophic failures of marine diesel engines: A case study. In: *Proceedings European Control Conference*, Thessaloniki, Greece, 24–27 June.
- Osen, O.L., 2016. Optimizing electric energy production on-board offshore vessels: Vessel power consumption profile and production strategies using genetic algorithms. In: *OCEANS 2016*, Shanghai, China.
- Park, Youngnam, Kim, Heemoon, 2024. Advanced design of naval ship propulsion systems utilizing battery-diesel generator hybrid electric propulsion systems. *J. Mar. Sci. Eng.* 12 (11), 2034.
- Pettersen, S.S., 2015. Design Flexible Offshore Construction Vessels to Handle Future Uncertainty (Msc Thesis). NTNU.
- Rubio, J.A.P., Vera-García, F., Grau, J.H., Cámara, J.M., Hernandez, D.A., 2018. Marine diesel engine failure simulator based on thermodynamic model. *Appl. Therm. Eng.* 144, 982–995.
- Sun, Jiawen, Ren, Hongxiang, Duan, Yating, Yang, Xiao, Wang, Delong, Tang, Haina, 2024. Fusion of multi-layer attention mechanisms and CNN-LSTM for fault prediction in marine diesel engines. *J. Mar. Sci. Eng.* 12 (6), 990.
- Susto, G.A., Schirru, A., Pampuri, S., McLoone, S., Beghi, A., 2015. Machine learning for predictive maintenance: A multiple classifiers approach. *IEEE Trans. Ind. Inform.*
- Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, Polosukhin, Illia, 2017. Attention is all you need.
- Wang, Zezhong, Zeng, Xingshan, Liu, Weiwei, Li, Liangyou, Wang, Yasheng, Shang, Lifeng, Jiang, Xin, Liu, Qun, Wong, Kam-Fai, 2025. Toolflow: Boosting llm tool-calling through natural and coherent dialogue synthesis. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. pp. 4246–4263.
- Yang, An, Li, Anfeng, Yang, Baosong, Zhang, Beichen, Hui, Binyuan, Zheng, Bo, Yu, Bowen, Gao, Chang, Huang, Chengen, Lv, Chenxu, Zheng, Chujie, Liu, Dayiheng, Zhou, Fan, Huang, Fei, Hu, Feng, Ge, Hao, Wei, Haoran, Lin, Huan, Tang, Jialong, Yang, Jian, Tu, Jianhong, Zhang, Jianwei, Yang, Jianxin, Yang, Jiayi, Zhou, Jing, Zhou, Jingren, Lin, Junyang, Dang, Kai, Bao, Keqin, Yang, Kexin, Yu, Le, Deng, Lianghao, Li, Mei, Xue, Mingfeng, Li, Mingze, Zhang, Pei, Wang, Peng, Zhu, Qin, Men, Rui, Gao, Ruize, Liu, Shixuan, Luo, Shuang, Li, Tianhao, Tang, Tianyi, Yin, Wenbiao, Ren, Xingzhang, Wang, Xinyu, Zhang, Xinyu, Ren, Xuancheng, Fan, Yang, Su, Yang, Zhang, Yichang, Zhang, Yinger, Wan, Yu, Liu, Yuqiong, Wang, Zekun, Cui, Zeyu, Zhang, Zhenru, Zhou, Zhipeng, Qiu, Zihan, 2025. Qwen3 technical report.
- Zhang, Peng, Gao, Zeyu, Cao, Lele, Dong, Fangyang, Zou, Yongjiu, Wang, Kai, Zhang, Yuesun, Sun, Peiting, 2022. Marine systems and equipment prognostics and health management: a systematic review from health condition monitoring to maintenance strategy. *Machines* 10 (2), 72.